



SRI KRISHNA COLLEGE OF ENGINEERING AND TECHNOLOGY

An Autonomous Institution| Approved by AICTE| Affiliated to Anna University
Kuniamuthur, Coimbatore - 641008

DEPARTMENT OF COMPUTER SCIENCE AND BUSINESS SYSTEMS

II B.Tech. – Computer Science and Business Systems

23CB402 – DATABASE FRAMEWORKS

PRACTICAL RECORD

Submitted by

Name :

Reg.No :

Branch :

Class/Section :

Semester :



SRI KRISHNA COLLEGE OF ENGINEERING AND TECHNOLOGY

An Autonomous Institution| Approved by AICTE| Affiliated to Anna University

Kuniamuthur, Coimbatore - 641008

DEPARTMENT OF COMPUTER SCIENCE AND BUSINESS SYSTEMS

II B.Tech. – Computer Science and Business Systems

PRACTICAL RECORD

Name:..... RegisterNo.:.....

Class/Semester:..... Degree & Branch:.....

BONAFIDE CERTIFICATE

This is to certify that this record is the bonafide record of work done by Mr./Ms _____
during the academic year 2024–2025.

Faculty In-charge

Head of the Department

Submitted for End Semester practical Examination held on.....

EXTERNAL EXAMINER

INTERNAL EXAMINER



SRI KRISHNA COLLEGE OF ENGINEERING AND TECHNOLOGY

An Autonomous Institution| Approved by AICTE| Affiliated to Anna University

Kuniamuthur, Coimbatore - 641008

DEPARTMENT OF COMPUTER SCIENCE AND BUSINESS SYSTEMS

II B.Tech. – Computer Science and Business Systems

Record of laboratory work

EVEN SEMESTER-2022-2023

CONTINUOUS EVALUATION SHEET

REFERENCES RUBRICS TABLE

Criteria	Range of Marks			
	Excellent	Good	Average	Below Average
Objective & Description with sample data (20)	18-20	17-18	15-16	0-14
Formation of SQLQuery PL/SQL Program (30)	27-30	21-26	15-20	0-14
Execution and Testing(30)	27-30	21-26	15-20	0-14
Documentation(10)	9-10	7-8	5-6	0-4
Viva(10)	9-10	7-8	5-6	0-4
Over all Marks	90-100	70-89	50-69	0-49

INDEX

Ex. No	Name of the Experiment	Page No.	Marks	Signature
1	Conceptual Database design using E-R DIAGRAM			
2	Implementation of SQL commands DDL, DML, DCL and TCL			
3	Queries to demonstrate implementation of Integrity Constraints			
4	Practice of Inbuilt functions			
5	Implementation of Joins, Nested Queries and Set operators			
6	Implementation of virtual tables using Views			
7	Practice of Procedural extensions			
8	Application Development using front end tools			
9	Document Database creation using MongoDB			
10	Study of Cloud Storage			
11	Mini Project (Application Development using DB)			



SRI KRISHNA COLLEGE OF ENGINEERING AND TECHNOLOGY
An Autonomous Institution| Approved by AICTE| Affiliated to Anna University
Kuniamuthur, Coimbatore - 641008

DEPARTMENT OF COMPUTER SCIENCE AND BUSINESS SYSTEMS

23CB402 – DATABASE FRAMEWORKS

Components	EX1	EX2	EX3	EX4	EX5	EX6	EX7	EX8	EX9	EX10
Objective & Description with sample data (20 Marks)										
Formation of SQL Query / PL/SQL Program (30 Marks)										
Execution and Testing (30 Marks)										
Documentation (10Marks)										
Viva (10 Marks)										
TOTAL										
AVERAGE										
Consolidated Mark(100)										
Faculty Signature										

Ex. No:1 **CONCEPTUAL DATABASE DESIGN USING E-R DIAGRAM**

AIM:

To create an conceptual database design using E-R diagram for an application

DESCRIPTION:

An entity-relationship diagram (ERD) is a data modeling technique that graphically illustrates an information system's entities and the relationships between those entities. An ERD is a conceptual and representational model of data used to represent the entity framework infrastructure.

The main elements of an ERD are:

- Entities
- Relationships
- Attributes

ENTITY:An entity is a real-world item or concept that exists on its own. Entities are equivalent to database tables in a relational database, with each row of the table representing an instance of that entity.

RELATIONSHIP:A relationship is the association that describes the interaction between entities.

TYPES:

1. Unary Relationship-Relationship between single entity
2. Binary Relationship-Relationship between two entity
3. Ternary Relationship-Relationship between three entity

ATTRIBUTE:An attribute of an entity is a particular property that describes the entity.

TYPES:

1. **Simple-** An attribute cannot be further subdivided(Eg:Reg no)
2. **Composite-**An attribute can be further subdivided(Eg:Name---Firstname,Middlename,Lastname)
3. **Single valued-**An attribute has only one value(Eg:Reg No)
4. **Multi valued-**An attribute has more than one value(Eg:Phone Number)
5. **Stored -** An attribute's value cannot be determined from the values of other attributes(Eg:Date of Birth)
6. **Derived-** An attribute's value can be determined from the values of other attributes(Eg:Age which is derived from Date of Birth)
7. **Descriptive-** Attributes of the relationship is called descriptive attribute.(Eg: employee works for department. Here 'works for' is the relation between employee and department entities. The relation 'works for' can have attribute DATE_OF_JOIN)

CONSTRAINTS:Contents of a database system must conform”

TYPES:

1. Mapping Cardinalities
2. Participation Cardinalities

MAPPING CARDINALITIES/CARDINALITY RATIO-Maximum no of relationship in which an entity can participate.

TYPES:

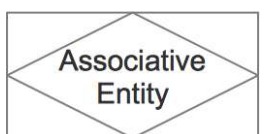
1. **One to One**-An entity in A is associated with at most one entity in B, and an entity in B is associated with at most one entity in A
2. **One to Many**-An entity in A is associated with any number of entities in B. An entity in B, however can be associated with at most one entity in A.
3. **Many to One**-An entity in A is associated with at most one entity in B. An entity in B, however can be associated with any number of entities in A.
4. **Many to Many**-An entity in A is associated with any number of entities in B, and an entity in B is associated with any number of entities in A

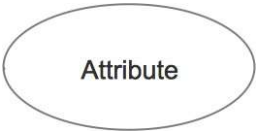
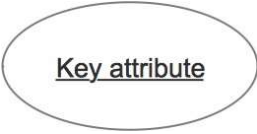
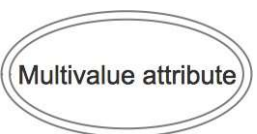
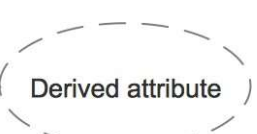
PARTICIPATION CONSTRAINTS/ EXISTENCE –Minimum no of relationship in which an entity can participate.

TYPES:

1. **Total Participation**-Every entity in E participates in at least one relationship in R
2. **Partial Participation**-Only some entities in E participate in relationships in R.

Entity Relationship Diagram Symbols — Chen notation

Symbol	Shape Name	Symbol Description
Entities		
	Entity	An entity is represented by a rectangle which contains the entity's name.
	Weak Entity	An entity that cannot be uniquely identified by its attributes alone. The existence of a weak entity is dependent upon another entity called the owner entity. The weak entity's identifier is a combination of the identifier of the owner entity and the partial key of the weak entity.
	Associative Entity	An entity used in a many-to-many relationship (represents an extra table). All relationships for the associative entity should be many
Attributes		

	Attribute	In the Chen notation, each attribute is represented by an oval containing attribute's name
	Key attribute	An attribute that uniquely identifies a particular entity. The name of a key attribute is underscored.
	Multivalued attribute	An attribute that can have many values (there are many distinct values entered for it in the same column of the table). Multivalued attribute is depicted by a dual oval.
	Derived attribute	An attribute whose value is calculated (derived) from other attributes. The derived attribute may or may not be physically stored in the database. In the Chen notation, this attribute is represented by dashed oval.
Relationships		
	Strong relationship	A relationship where entity is existence-independent of other entities, and PK of Child doesn't contain PK component of Parent Entity. A strong relationship is represented by a single rhombus
	Weak (identifying) relationship	A relationship where Child entity is existence-dependent on parent, and PK of Child Entity contains PK component of Parent Entity. This relationship is represented by a double rhombus.

STEPS TO DESIGN AN ER DIAGRAM:

Step 1: Identify the entities from the requirements gathered.

- Strong Entity
 - Doctor
 - Patient
- Weak Entity
 - Specialization

- Treatment

Step 2: Finding the relationship between those entities.

- Treats:
 - Between Doctor and Patient
- Has:
 - Between Doctor and Specialization
- Takes:
 - Between Patient and Treatment

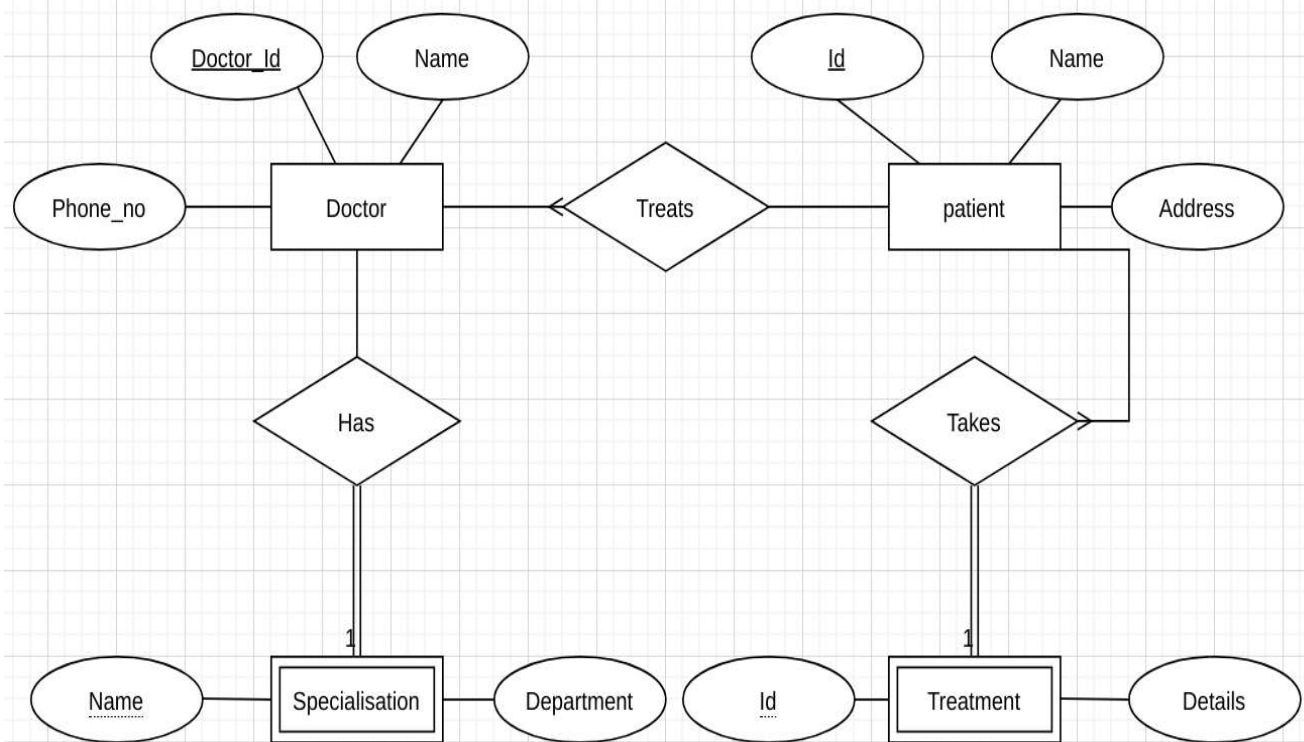
Step 3: Identify the key attributes for all entities.

- Doctor - Doctor_Id
- Patient - Id
- Specialization - Name
- Treatment - Id

Step 4: Identify the other relevant attributes.

- Doctor:
 - Phone_no
 - Name
- Patient:
 - Name
 - Address
- Specialization:
 - Department
- Treatment:
 - Details

Step 5: Draw an ER diagram.



Result:

Thus the implementation of E-R diagram for has been designed successfully.

Exp No: 2 IMPLEMENTATION OF SQL COMMANDS DDL, DML, DCL AND TCL

DATA DEFINITION LANGUAGE COMMANDS

OBJECTIVES

To implement the various Data Definition Language (DDL) commands using SQL.

LEARNING OUTCOMES

After the completion of this experiment, student will be able to

- Create a table and understand the table structure to perform any SQL operations
- Alter a table to add columns or modify the table data types for storing various data.
Delete all contents of the table and drop the table and realize the importance of DDL.

PROBLEM STATEMENT

The queries to implement Data Definition Language (DDL) commands are

1. Create
2. Desc
3. Alter (add, modify)
4. Truncate
5. Drop

DESCRIPTION

1. Create Table

This command is used to create a new table in RDBMS.

Syntax

Create table<table name> (column definition1, column definition2,...);

2. Desc Table

This command is used to view the structure of the table.

Syntax

Desc<table name>;

3. Alter Table

This command is used to change the structure of a table. Alter command is used to:

- a. Add a new column.
- b. Modify the existing column definition

Syntax

- a. Alter table <table name> modify (column definition..);
- b. Alter table<table name> add (column definition..);

4. Truncate Table

This command is used to delete all records from the table but the structure of the table will be retained.

Syntax

Truncate table <table name>;

5. Drop Table

This command is used to delete the table structure.

Syntax

Drop table <table name>;

TEST CASES:

1. From the Table DDL of employees table given below, Write a query to create a table **employees** .

Note:

1. Make sure to apply **NOT NULL** and Primary Key (PK) condition for the columns where ever applicable.
2. While creating a table, table name and column name should be exactly same as given in DDL (including upper/lower case)

Table name: employees

Column Name	Data Type	Can be Nullable?
employee_id (PK)	int	No
first_name	varchar(25)	No
last_name	varchar(25)	No
email	varchar(25)	No
phone_number	varchar(20)	No
hire_date	date	No
job_id	varchar(10)	Yes
salary	decimal(8,2)	No
commission_pct	int	Yes
manager_id	int	Yes
department_id	int	Yes

Solution:

create table employees(employee_id int Primary key ,first_name varchar(25) not null,last_name varchar(25) not null,email varchar(25) not null,phone_number varchar(20) not null,hire_date date not null,job_id varchar(10),salary decimal(8,2) not null,commission_pct int,manager_id int,department_id int);

2. From the DDL given below,
Write a query to create a table "**facilities**"

Hint: Refer the table for the field names and its data type of the table.

Table name: facilities

Column Name	Data Type	Can be Nullable?
facid(PK)	int	No
name	varchar(100)	No
membercost	int	No
guestcost	int	No
initialoutlay	int	Yes
monthlymaintenance	int	Yes

Note:

1. Make sure to apply NOT NULL and Primary Key (PK) condition for the columns where ever applicable.
2. While creating a table, table name and column name should be exactly same as given in DDL (including upper/lower case)

Solution:

create table facilities(facid int primary key,name varchar(100) not null,membercost int not null,guestcost int not null,initialoutlay int,monthlymaintenance int);

3. Write a query to drop the table **facilities**

Solution:

drop table facilities;

4. Write a query to drop the table **facilities**

Solution:

drop table facilities;

5. From the DDL given below,

Write a query to change the column name zipcode to pincode in members table.

Table name: members

Column Name	Data Type	Can be Nullable?
memid(PK)	int	No
surname	varchar(200)	No
firstname	varchar(200)	No
address	varchar(300)	No
zipcode	int	No
telephone	varchar(20)	No
recommendedby	int	Yes
joindate	varchar(10)	No

Solution:

alter table members change column zipcode pincode int;

RESULT:

Thus the Data Definition Language (DDL) commands are executed successfully.

OBJECTIVES

To implement the various Data Manipulation Language (DML) commands using SQL.

LEARNING OUTCOMES

After the completion of this experiment, student will be able to

- Query and manipulate the existing data base objects in table structure.
- Insert into a table to add rows or modify the table to update values for storing data.
- Delete the unnecessary contents of the table, rename it and display the reformed table.

PROBLEM STATEMENT

The queries to implement the Data Manipulation Language (DML) commands are

1. insert
2. select
3. update
4. delete
5. rename

DESCRIPTION

1. Insert Command

This is used to add one or more rows to a table.

a. Inserting Values Into Table for All Fields

Here the number of values must correspond to number of column in table.

Syntax

Insert into <table name> values (value1, value2, value3...);

b. Inserting Values From User

Here the values of the columns in table are obtained from user during run-time.

Syntax

Insert into <table name> values (<&column1>, <&column2>...);

2. Select Command

This is used to retrieve columns from the table.

a. Selecting all rows from the table

Syntax

Select * from <table name>;

b. Selecting specific columns from the table

Syntax

Select column_name1,,column_nameN from <table name>;

c. Selecting rows using where clause from the table

Syntax

Select column_name1,,column_nameN from <table name>;

3. Update Command

This is used to modify one or a set of rows at a time.

Syntax

Update <table name> set <col1> = value1, <col2> = value2...where <condition>;

4. Delete Command

This is used to remove the row from the table..

Syntax

Delete from <table name> where <condition>;

SAMPLE TEST CASES

1. Write a query to insert any 4 records to the table 'employees'
The column names and data types are given below.

Column Name	Data type
EMPLOYEE_ID(PK)	INT
FIRST_NAME	VARCHAR (20)
LAST_NAME	VARCHAR (25)
EMAIL	VARCHAR (25)
PHONE_NUMBER	VARCHAR (20)
HIRE_DATE	DATE
JOB_ID	VARCHAR (10)
SALARY	DECIMAL (8, 2)
COMMISSION_PCT	INT
MANAGER_ID	INT
DEPARTMENT_ID	INT

Solution:

INSERT INTO employees VALUES

```
(1,'A','B','GG@GMAIL.COM','123','2021-03-02','A',100,1,2,3),  
(2,'C','D','GG2@GMAIL.COM','23','2021-03-05','B',200,4,5,6),  
(3,'B','V','GG3@GMAIL.COM','24','2022-03-05','C',300,34,35,36),  
(4,'V','VV','GG4@GMAIL.COM','34','2022-06-05','G',400,11,22,33);
```

2. Write a query to update manager id to 100 for department id 95.

Table details are as follows:

Table Name: Employees (Table names are case sensitive)

Column Names (along with data types)

employee_id - INT *Primary key*,

first_name - VARCHAR(30), last_name - VARCHAR(30),

email - VARCHAR(30), phone_number - VARCHAR(20),

hire_date - DATE,

job_id - VARCHAR(10),

salary - DECIMAL(18,2),

manager_id - INT,

department_id - INT

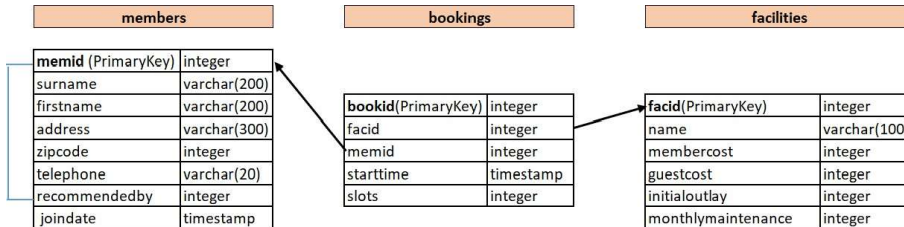
Solution:

update Employees set manager_id=100 where department_id=95;

3. Write a query to update facility name as 'Snooker Table-2' instead of facility name 'Pool Table' in facilities table.

name column in facilities table refers to facilities name.

Refer the table details below:



Solution:

**update facilities set name ='Snooker Table-2'
where name ='Pool Table';**

4. Write a query to delete all the records from 'members' table

Solution:

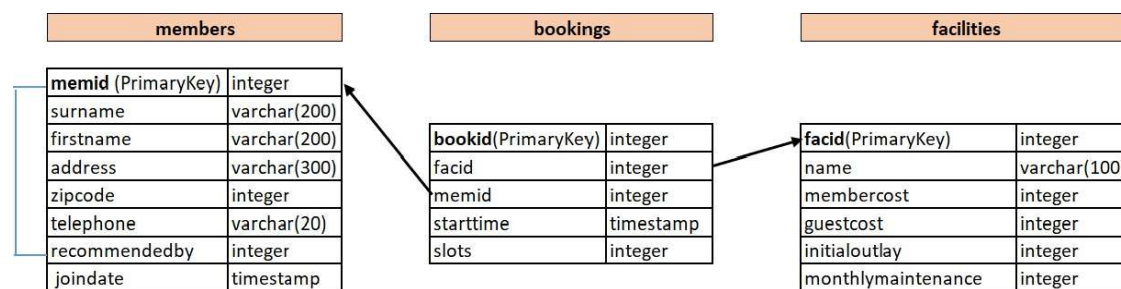
truncate table members;

5. Write a query select the 'joindate' from the table 'members' where the memid is even.

Solution:

select joindate from members where memid %2 = 0;

6. Write a query update the column membercost to 1000 from the table facilities where the monthlymaintenance is greater than 500.



Solution:

update facilities set membercost=1000 where monthlymaintenance>500;

RESULT:

Thus the Data Manipulation Language (DML) commands are executed successfully.

OBJECTIVES

To implement the various Data Control Language (DCL) and Transaction Control Language (TCL) commands using SQL.

LEARNING OUTCOMES

After the completion of this experiment, student will be able to

- Perform transactions with the existing data base objects in table.
- Undo, redo and save the operations done with the existing rows in the table.
- Understand the access control of database, user privileges and securing the database.

PROBLEM STATEMENT

The queries to implement Data Control Language (DCL) and Transaction Control Language (TCL) commands are

1. grant
2. revoke
3. commit
4. save point
5. roll back

DESCRIPTION

DCL Commands

1. Grant Command

This statement will grant permissions for any particular operation.

Syntax

Grant <database_priv [database_priv.....] >to<user_name> identified by <password>

[,<password.....];

2. Revoke Privilege Command

Using this statement the privilege that you specify to user are revoked cascade required to remove any referential integrity constraints.

Syntax

Revoke <database_priv> from <user / public>;

TCL COMMANDS

1. Commit Command

It completes the transactions done by making changes permanently to the database.

Syntax

Commit;

2. Save point Command

Save points are temporary markers that are used to divide a lengthy transaction and save the changes made as each point of reference to the table.

Syntax

Save point<save point id>;

3. Rollback Command

It reverts back the changes made and completes the transaction in the table.

Syntax

Rollback; (or) rollback to <save point id>;

SAMPLE CODING AND EXECUTION OF THE PROGRAM

1. SQL> Grant select, update , insert on employees to departments with grant option;
2. SQL> Revoke select, update , insert on employees from departments;
3. SQL> commit;
4. SQL> savepoint ep;
5. SQL> rollback to savepoint ep1;

SAMPLE OUTPUT

DCL queries:

Creation of a new user & Query to view the list of users:

```
mysql> select user from mysql.user;
+-----+
| user |
+-----+
| ashwin09 |
| mysql.infoschema |
| mysql.session |
| mysql.sys |
| root |
+-----+
5 rows in set (0.00 sec)

mysql> create user sample@localhost identified by 'ash';
Query OK, 0 rows affected (0.05 sec)
```

GRANT

Accessing root user database (from newly created user sample@localhost) before granting access:

```
mysql> use jason;
ERROR 1044 (42000): Access denied for user 'ashwin09'@'localhost' to database 'jason'
```

Creating Database in root user, sample table and granting access.

```
mysql> select * from sample;
+----+-----+
| id | name |
+----+-----+
| 1  | Ashwin |
| 1  | Jason |
| 2  | Barath |
| 3  | Aswin |
+----+-----+
4 rows in set (0.00 sec)
```

```
mysql> grant select on jason.sample to sample@localhost;
Query OK, 0 rows affected (0.01 sec)
```

Accessing sample user after granting access:

```
mysql> use jason;
Database changed
mysql> select * from sample;
+-----+-----+
| id | name |
+-----+-----+
| 1 | Ashwin |
| 1 | Jason |
| 2 | Barath |
| 3 | Aswin |
+-----+-----+
4 rows in set (0.00 sec)

mysql> insert into sample values(5,'Dhaya');
ERROR 1142 (42000): INSERT command denied to user 'ashwin09'@'localhost' for table 'sample'
mysql> exit
```

REVOKE

Revoke access from sample:

```
mysql> revoke select on jason.sample from sample@localhost;
Query OK, 0 rows affected (0.01 sec)

mysql> _
```

Accessing after revoke:

```
mysql> use jason;
ERROR 1044 (42000): Access denied for user 'sample'@'localhost' to database 'jason'
mysql> _
```

Commit & Rollback:

```
+-----+-----+
| ID | NAME |
+-----+-----+
| 18 | Ashwin |
| 53 | Jason |
| 20 | Aswin |
| 23 | Barath |
+-----+-----+
4 rows in set (0.01 sec)

mysql> Start transaction;
Query OK, 0 rows affected (0.00 sec)

mysql> set autocommit=0;
Query OK, 0 rows affected (0.00 sec)

mysql> commit;
Query OK, 0 rows affected (0.00 sec)

mysql> insert into sample values(04,'Ajay');
Query OK, 1 row affected (0.00 sec)

mysql> Select * from sample;
+-----+-----+
| ID | NAME |
+-----+-----+
| 18 | Ashwin |
| 53 | Jason |
| 20 | Aswin |
| 23 | Barath |
| 4 | Ajay |
+-----+-----+
5 rows in set (0.00 sec)
```

```
mysql> rollback;
Query OK, 0 rows affected (0.00 sec)

mysql> Select * from sample;
+-----+
| ID | NAME |
+-----+
| 18 | Ashwin |
| 53 | Jason |
| 20 | Aswin |
| 23 | Barath |
+-----+
4 rows in set (0.00 sec)
```

Save point:

```
mysql> savepoint A;
Query OK, 0 rows affected (0.00 sec)

mysql> update sample set name='Ajay' where id=23;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> Select * from sample;
+-----+
| ID | NAME |
+-----+
| 18 | Ashwin |
| 53 | Jason |
| 20 | Aswin |
| 23 | Ajay |
+-----+
4 rows in set (0.00 sec)

mysql> savepoint B;
Query OK, 0 rows affected (0.00 sec)

mysql> rollback to A;
Query OK, 0 rows affected (0.00 sec)

mysql> rollback to A;
Query OK, 0 rows affected (0.00 sec)

mysql> Select * from sample;
+-----+
| ID | NAME |
+-----+
| 18 | Ashwin |
| 53 | Jason |
| 20 | Aswin |
| 23 | Barath |
+-----+
```

RESULT:

Thus the Data Control Language (DCL) and Transaction Control Language (TCL) commands are executed successfully.

Exp. No: 03

Date:

QUERIES TO DEMONSTRATE IMPLEMENTATION OF INTEGRITY CONSTRAINTS

Aim

To implement various Integrity Constraints in SQL queries.

Description:

INTEGRITY CONSTRAINT

An integrity constraint is a mechanism used by oracle to prevent invalid data entry into the table. It has enforced the rules for the columns in a table.

The types of the integrity constraints are:

- a) Domain Integrity
- b) Entity Integrity
- c) Referential Integrity

a) Domain Integrity:

This constraint sets a range and any violations that take place will prevent the user from performing the manipulation that caused the breach.

It includes:

➤ Not Null constraint:

- While creating tables, by default the rows can have null value. The enforcement of not null constraints in a table ensures that the table contains values. Principle of null values:
- Setting a null value is appropriate when the actual value is unknown, or when a value would not be meaningful.
- A null value is not equivalent to a value of zero. A null value will always evaluate to null in any expression.
- When a column name is defined as not null, that column becomes mandatory i.e., the user has to enter data into it.
- Not null Integrity constraint cannot be defined using the alter table command when the table contains rows.

Syntax

- Column name data type[size] <not null>;

Example:

- Create table cust(custid number(6) not null, name char(10));
- Alter table cust modify (name not null);

➤ Check Constraint:

- Check constraint can be defined to allow only a particular range of values .when the manipulation violates this constraint, the record will be rejected. Check conditions cannot contain subqueries.

Syntax

Column name data type[size] <check><logical expression>.

Example:

- Create table student (regno number (6), mark number (3) constraint b check (mark >=0 and mark <=100));
- Alter table student add constraint b2 check (length(regno)<=4));

b) Entity Integrity:

Entity Integrity maintains uniqueness in a record.

An entity represents a table and each row of a table represents an instance of that entity. To identify each row in a table uniquely we need to use this constraint.

There are 2 entity constraints:

➤ **Unique key constraint:**

It is used to ensure that information in the column for each record is unique, as with telephone or drivers license numbers. It prevents the duplication of value with rows of a specified column in a set of columns. A column defined with the constraint can allow null value. If a unique key constraint is defined in more than one column i.e., combination of columns cannot be specified. Maximum combination of columns that a composite unique key can contain is 16.

Syntax

Column name data type[size] <unique>;

Example: Create table cust(custid number(6) constraint uni unique, name char(10));

Alter table custadd(constraint c unique(custid));

➤ **Primary Key Constraint:**

A primary key avoids duplication of rows and does not allow null values. It can be defined on one or more columns in a table and is used to uniquely identify each row in a table. These values should never be changed and should never be null. A table should have only one primary key. If a primary key constraint is assigned to more than one column or combination of columns is said to be a composite primary key, which can contain 16 columns.

Syntax

Column name data type [size] <primary key>;

Example: Create table stud(regno number(6) constraint primary key, name char(20));

c) Referential Integrity:

It enforces relationships between tables.

To establish parent-child relationship between 2 tables having a common column definition, we make use of this constraint.

To implement this, we should define the column in the parent table as primary key and the same column in the child table as foreign key referring to the corresponding parent entry.

Foreign key: A column or combination of columns included in the definition of referential integrity, which would refer to a referenced key.

Referenced key: It is a unique or primary key upon which is defined on a column belonging to the parent table.

Syntax

Column name data type[size] <references> TABLE1 NAME(column name of table 1);

Example:

PRIMARY KEY TABLE:

```
CREATE TABLE product( product_id number(5) CONSTRAINT pd_id_pk PRIMARY KEY,  
product_name char(20),supplier_name char(20),unit_price number(10));
```

FOREIGN KEY TABLE:

```
CREATE TABLE order_items  
( order_id number(5) CONSTRAINT od_id_pk PRIMARY KEY,  
product_id number(5) CONSTRAINT pd_id_fk REFERENCES  
product(product_id),  
product_name char(20),  
supplier_name char(20),  
unit_price number(10));
```

Or

```
CREATE TABLE order_items  
( order_id number(5) ,  
product_id number(5),  
product_name char(20),  
supplier_name char(20),  
unit_price number(10)  
CONSTRAINT od_id_pk PRIMARY KEY(order_id),  
CONSTRAINT pd_id_fk FOREIGN KEY(product_id) REFERENCES product(product_id));
```

SAMPLE TEST CASES:

1. From the table schema given below, write a query to create a table orders.
(Follow the same table names/column names as given along with the case)
The customer table is already created with primary key as customerNumber

Note:

In orders table, orderNumber is the Primary key and customerNumber is the Foreign Key referencing customerNumber of customers table.

While creating orders table,

orderNumber is the Primary Key

define the foreign key relation between 2 tables.

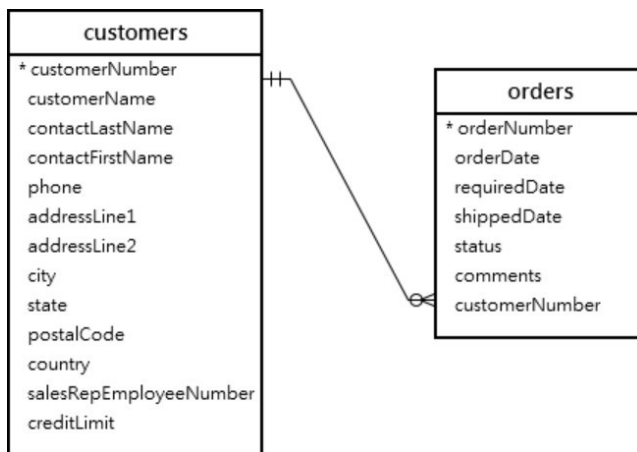
define all the columns with NOT NULL condition

Follow the below data type:

orderNumber (int), orderDate(date), requiredDate(date), shippedDate(date), status(varchar(15)),

comments(text),

customerNumber(int)



Solution:

```
create table orders(orderNumber int primary key,  
orderDate date not null,  
requiredDate date not null,  
shippedDate date not null,  
status varchar(15) not null,  
comments text not null,  
customerNumber int,  
foreign key(customerNumber) references customers(customerNumber));
```

2. Adding Foreign key Constraint in the existing table

We have created an employee and department tables with the below definition without adding foreign key relation

Table name: employee

Column Name	Data Type
id (PK)	int
name	varchar(100)
address	varchar(100)
age	int
dob	date
deptId	int

Table name: department

Column Name	Data Type
departmentId (PK)	int
departmentName	varchar(100)

We want to link departmentId (department table) and deptId column (employee table) to maintain data integrity by establishing foreign key relation.

Write a alter query to employee table so that deptId will be foreign key referencing departmentId of department table.
with reference name as depIdFk.

Solution:

alter table employee

add constraint depIdFk foreign key(deptId) references department(departmentId);

Result:

Thus, the various integrity constraints queries are executed successfully in SQL.

Ex.No: 04

INBUILT FUNCTIONS

Aim:

To perform Inbuilt functions in SQL.

Learning Outcomes:

After the completion of this experiment, student will be able to

- Perform mathematical calculations, character functions and other functions.
- Understand the need for pre-defined functions in real time scenarios.

Problem Statement:

SQL has many built-in functions for performing calculations on data.

1. Mathematical functions
2. Character functions
3. Conversion Function
4. Group Function (Aggregate function)
5. Miscellaneous Function
6. Special Function

Description:

1. Mathematical functions:

There are many mathematical functions. All mathematical functions return NULL in the event of an error. Some of the mathematical functions are:

- a. ABS(X)-Returns the absolute value of X.
- b. CEIL(X)-Returns the smallest integer value not less than X.

2. Character functions

A character function is a function that takes one or more-character values as parameters and returns either a character value or a number value. Some of the character functions are:

- a. RTRIM-Trims the right side of a string of all specified characters.
- b. LPAD-Pads a string on the left with the specified characters.

3. Conversion Functions:

Conversion functions are used to must first convert the data to be the right datatype for the operation. Some of the conversion functions are:

- a. ADD_MONTHS (date, n) - Returns a date value after adding 'n' months to the date 'x'.
- b. LAST_DAY (x) -It is used to determine the number of days remaining in a month from the date 'x' specified.
- c. NVL-The Oracle NVL function lets you substitute a value when a null value is encountered.

EG: NVL(string1, replace_with)

4. Group Function (Aggregate function):

- a. MAX(): This function is used to get the maximum value from a column.
- b. AVG(): This function is used to get the average value of a numeric column.

5. Miscellaneous Function:

a. **UID function:** It returns an integer that uniquely identifies the current user.

SAMPLE TEST CASES:

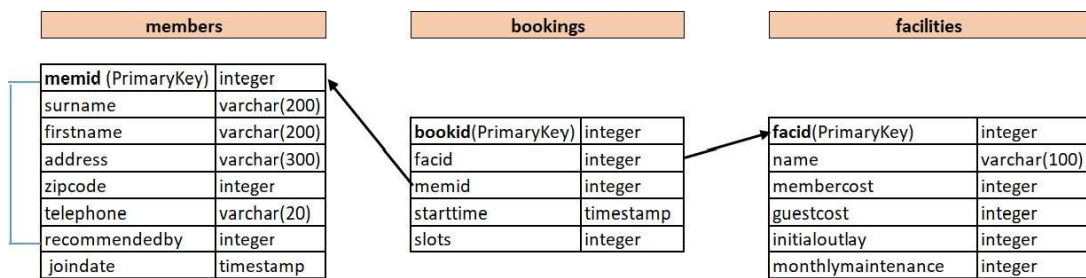
1. From the Schema given below, write a query to produce a count of the number of recommendations each member has made. Sort the output by ascending order of memid (recommendedby).

If a particular member did not recommend anyone, ignore that memid. Output *recommendedby* and the corresponding count.

Note:

Refer table members from the given Schema.

1. 'recommendedby' field denotes 'memid' who recommended that particular member in the members table.
2. Table names are case sensitive.
3. Refer Output Format section for the output header names.



Solution:

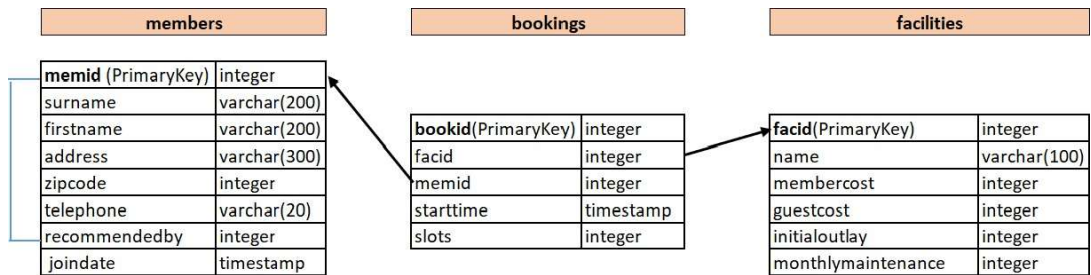
```
select recommendedby,count(*)
as count
from members
where recommendedby is not null
group by recommendedby
order by recommendedby;
```

2. From the Schema given below, write a query to produce a list of the total number of slots booked per facility per month for July and August 2012. (month should be fetched as 7 and 8 corresponding to July and August)

The output should have the details of facility id (facid), month, total slots booked sorted by the facid and month in ascending order.

Note:

1. Refer table bookings from the given Schema.
2. Refer the field 'starttime' to filter month
3. Table names are case sensitive.
4. Refer Output Format section for the output header names.



Solution:

```

select facid,extract(month from starttime)as month,sum(slots)
as 'Total Slots' from bookings
where
starttime>='2012-07-01' and starttime<'2012-09-01'
group by facid,month
order by facid,month;

```

3. Given a table with order details. Please find the table details and sample data below.

TABLE NAME: AGG_ORDERS

FIELD NAMES: ord_no, purchase_amt, ord_date, cust_id, salesman_id

ord_no	purchase_amt	ord_date	cust_id	salesman_id
20001	788.5	15-08-2019	123	5000
20005	975.5	15-01-2019	278	5012

Write a SQL statement to find the highest purchase amount with their Customer ID and order date, for only those customers who have the highest purchase amount in a day equal to any of these values 1000, 2200, 3700, 4000. Sort the records in ascending order of Customer ID.

Note: If a customer has done 2 purchases on a same day with purchase amount falls in our list (1000, 2200, 3700, 4000), then output should display both the records.Group by ord_no,cust_id,ord_date in same order.

Solution:

```

select cust_id as Customer_ID,ord_date as Date ,max(purchase_amt) as Amount
from AGG_ORDERS
group by
ord_no,cust_id,ord_date
having max(purchase_amt)in(1000,2200,3700,4000)
order by cust_id;

```

Result:

Thus, the built-in functions in SQL are executed successfully.

EX NO: 5**JOINS, NESTED QUERIES AND SET OPERATORS****AIM:**

To Implement Joins and Nested Queries and Set operators

LEARNIG OUTCOMES:

After the completion of this experiment,students will be able to

- Perform Nested queries operations.
- Perform Join Operations
- To utilize various set operators available in MySQL.

TEST CASES:**JOINS:**

1.You are given two tables of XYZ company, EMPLOYEE_XYZ and INCENTIVES_XYZ which has details about employees of a company and their incentives. Write a SQL Query that fetches the employee Id as ID, first name of the employees as First_name and sum of the incentives as Incentive for each IDs from the Incentives table.

Display in ascending order of employee ID.

Table names are case sensitive.

TABLE 1: EMPLOYEE_XYZ

Column Names: Employee_id, First_name,Last_name,Salary,Joining_date,Department

TABLE 2: INCENTIVES_XYZ

Column Names: Id, Incentive_date,Incentive_amount

Below are only sample tables with sample data. Original data will be populated in the back end.

TABLE 1: EMPLOYEE_XYZ

Employee_id	First_name	Last_name	Salary	Joining_date	Department
1	John	Abraham	1000000	11-10-2008	Banking
2	Michael	Clarke	800000	11-09-2007	Insurance
3	Roy	Thomas	700000	11-08-2006	Banking
4	Tom	Jose	600000	22-11-2008	Insurance

TABLE 2: INCENTIVES_XYZ

Id	Incentive_date	Incentive_amount
1	01-02-2013	5000
2	01-02-2013	3000

Solution:

```
select Employee_id as ID ,First_name as First_name,sum(incentive_amount) as Incentive
from EMPLOYEE_XYZ a join INCENTIVES_XYZ b on a.Employee_id=b.id group by ID;
```

2. Dave owns a book shop. He is planning to stock all the best selling books in his shop. He collected all the information on author, book names and the number of copies sold in the below format.

Table1 Details: Table names are case sensitive

Table Name: authorBooks

Column Name: authorName, bookName

authorName	bookName
Daniel Pink	Drive
Leo Tolstoy	War and Peace
William Shakespeare	Hamlet
William Shakespeare	Othello
J.K.Rowling	Harry Potter
Agatha Christie	The Mouse Trap
Agatha Christie	And Then There Were None

Table2 Details:

Table Name: soldCopies

Column Name: bookName, soldCopies

bookName	soldCopies
Drive	30000
War and Peace	200000
Hamlet	500000
Othello	400000
Harry Potter	700000
The Mouse Trap	80000
And Then There Were None	100000

Create an SQL query that shows the top 3 authors who sold the most books in total along with total number of copies sold.

Sort the output in descending order of sold copies.

The headers in the output must be named as follows

Author_Name sold_sum

Solution:

```
SELECT authorBooks.authorName AS Author_Name, SUM(soldCopies.soldCopies) AS
sold_sum
FROM authorBooks
```

JOIN soldCopies
ON authorBooks.bookName = soldCopies.bookName
GROUP BY authorBooks.authorName
ORDER BY sold_sum DESC LIMIT 3;

3. Here is the sample employee and salary information of ABC Infotech Limited company. The company has total of 30 employees.

Below are only the sample tables. The original table will be populated in the backend

Table names are case sensitive

Table Name: EMPLOYEES_ABC

Column Names: employeeID, employeeName, departmentName

employeeID	employeeName	departmentName
1	John	Management
2	Smith	Sales
3	Dave	Management
4	Sebolt	Business Analyst
5	Francis	Systems Engineer

Table Name: SALARIES_ABC

Column Names: employeeID, employeeName, Salary

employeeID	employeeName	Salary
1	John	10000
2	Smith	8000
3	Dave	15000
4	Sebolt	12000
5	Francis	7000

Write a query to display every department name where the average salary per employee is lower than 7000.

The headers in the output must be named as follows

Department_Name, Avg_salaries

Solution:

SELECT EMPLOYEES_ABC.departmentName AS Department_Name,
AVG(SALARIES_ABC.Salary) AS Avg_salaries
FROM EMPLOYEES_ABC
JOIN SALARIES_ABC
ON EMPLOYEES_ABC.employeeID = SALARIES_ABC.employeeID
GROUP BY departmentName HAVING AVG(SALARIES_ABC.Salary) < 7000

NESTED SUBQUERIES:

4. Suppose that a website contains two tables, the Customers table, and the Orders table. Write a SQL query to find all customers who never order anything.

Sort the result by ascending order of Customer Names

Table Details

Table1:

Tablename: CUSTOMERS

Columnname: id, cust_name

```
+-- +-----+
| id | cust_name|
+-- +-----+
| 1 | Prasanth |
| 2 | Harish   |
| 3 | Sam      |
| 4 | Nadhini  |
+-- +-----+
```

Table2:

Table name: ORDERS

Column name: id, customer_id

```
+-- +-----+
| id| customer_id|
+-- +-----+
| 1 | 3   |
| 2 | 1   |
+-- +-----+
```

Sample output

```
+-----+
| Customers |
+-----+
| Harish    |
| Nandhini  |
+-----+
```

Solution:

```
SELECT cust_name as 'Customers'
from CUSTOMERS
where id not in
(select customer_id from ORDERS)ORDER BY 1;
```


5. Write a query to display the name (first name and last name) for those employees who get more salary than the employee whose ID is 105.

Table details are as follows:

Table Name: EMPLOYEES_SUBQ (Table names are case sensitive)

Column Names (along with data types)

employee_id - INT *Primary key*,
first_name - VARCHAR(30), last_name - VARCHAR(30),
email - VARCHAR(30), phone_number - VARCHAR(20),
hire_date - DATE,
job_id - VARCHAR(10),
salary - DECIMAL(18,2),
manager_id - INT,
department_id - INT

Solution:

```
SELECT first_name, last_name  
FROM EMPLOYEES_SUBQ  
WHERE salary >  
( SELECT salary  
FROM EMPLOYEES_SUBQ  
WHERE employee_id=105  
);
```

6. Write a query to display the name (first name and last name), salary, department id, job id for those employees who work in the same designation(job id) as the employee works whose id is 110.

Table details are as follows:

Table Name: EMPLOYEES_SUBQ (Table names are case sensitive)

Column Names (along with data types)

employee_id - INT *Primary key*,
first_name - VARCHAR(30), last_name - VARCHAR(30),
email - VARCHAR(30), phone_number - VARCHAR(20),
hire_date - DATE,
job_id - VARCHAR(10),
salary - DECIMAL(18,2),
manager_id - INT,
department_id - INT

Solution:

```
SELECT first_name, last_name, salary, department_id, job_id
FROM EMPLOYEES_SUBQ
WHERE job_id =
( SELECT job_id
FROM EMPLOYEES_SUBQ
WHERE employee_id= 110
);
```

7. Write a query to find the products whose list price is equal to the highest list price of the products within the same category using Correlated Subquery concept.

Table Details: (table names are case sensitive)

Table Name: PRODUCTS

Column Names:

product_id INT PRIMARY KEY,
product_name VARCHAR (255) NOT NULL,
brand_id INT NOT NULL,
category_id INT NOT NULL,
model_year SMALLINT NOT NULL,
list_price DECIMAL (10, 2) NOT NULL

Sort the output in ascending order of category_id and product_name.

Solution:

```
SELECT
    product_name,
    list_price,
    category_id
FROM
    PRODUCTS p1
WHERE
    list_price IN (
        SELECT
            MAX(p2.list_price)
        FROM
            PRODUCTS p2
        WHERE
            p2.category_id = p1.category_id
        GROUP BY
            p2.category_id
    )
ORDER BY
```

**category_id,
product_name;**

8. Write a query to find the products(product_name,list_price) whose list prices(list_price) are greater than or equal to the average list price of any product brand (brand_id). Use Subquery concept.

Sort the output by ascending order of product_name.

Table Details:

Table Name: PRODUCTS

Column Names:

product_id INT *PRIMARY KEY*,
product_name VARCHAR (255) NOT NULL,
brand_id INT NOT NULL,
category_id INT NOT NULL,
model_year SMALLINT NOT NULL,
list_price DECIMAL (10, 2) NOT NULL

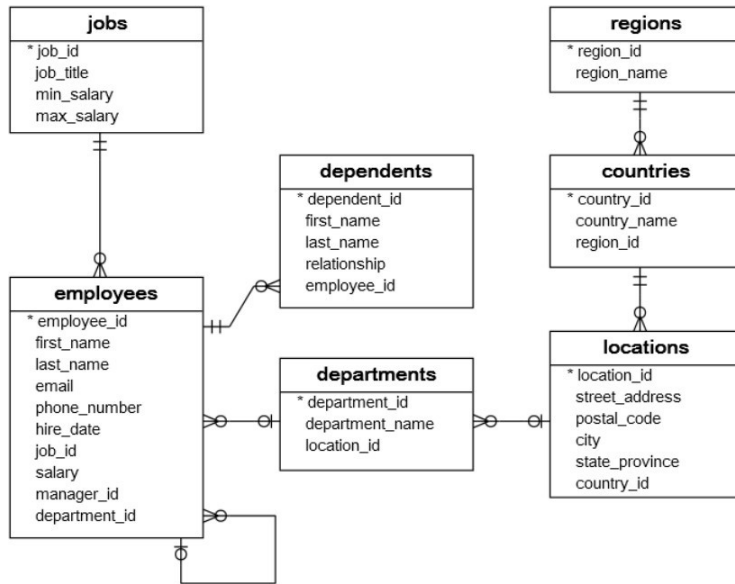
Solution:

```
SELECT
    product_name,
    list_price
FROM
    PRODUCTS
WHERE
    list_price >= ANY (
        SELECT
            AVG (list_price)
        FROM
            PRODUCTS
        GROUP BY
            brand_id
    )
order by product_name;
```

9. From the below ER diagram of HR database, Write a query to find all the employees (employee_id, first_name, last_name, salary, department_id) whose salary is higher than the average salary of the employees in their departments. Use Correlated Subquery concept.

Refer the tables employees from the below diagram for the column names (Table names are case sensitive)

Sort the output by ascending order of department_id, first_name, last_name.



Solution:

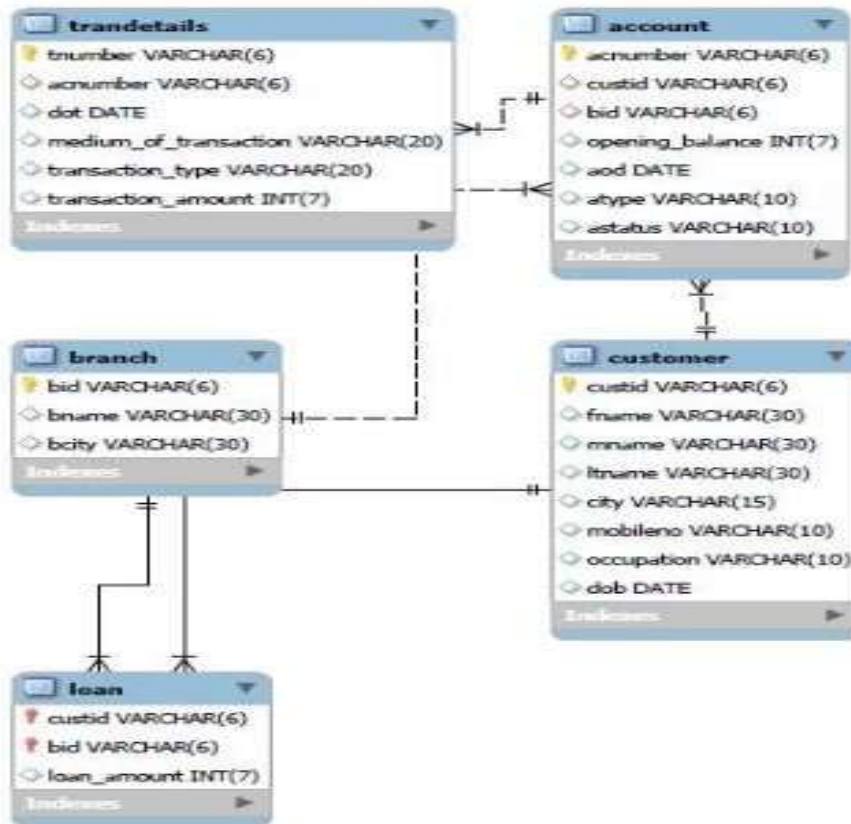
```

SELECT
    employee_id,
    first_name,
    last_name,
    salary,
    department_id
FROM
    employees e
WHERE
    salary > (SELECT
                AVG(salary)
            FROM
                employees
            WHERE
                department_id = e.department_id)
ORDER BY
    department_id, first_name, last_name
  
```

SET OPERATORS:

10. Given the schema of a Bank Management system, Write a query to display the customer's number, first name, and middle name.

The customer's who don't have a middle name, for them display the last name. Give the alias name as Cust_Name.



Note:

The required input tables are populated in the back end.

The table names are case-sensitive.

Refer to the schema and the output section for further clarifications.

Input format

No console input.

Output format

The output prints the custid and Cust_Name.

Solution:

```
select custid,concat(fname,' ',lname) as Cust_Name from customer where mname IS NULL
```

UNION

```
select custid,concat(fname,' ',mname) as Cust_Name from customer where mname!='NULL' order by custid;
```

RESULT:

Thus, the above experiment has been executed successfully.

EX.NO: 6

Date:

IMPLEMENTATION OF VIRTUAL TABLES USING VIEWS

Objectives:

1. To implement VIEW operations using SQL.

Learning Outcomes:

After the completion of this experiment, student will be able to

- Understand the use of virtual table and provide a restricted view to the data in the table.
- Summarize data from various tables which can be used to generate reports.

Problem Statement:

To perform VIEW the following commands are executed:

1. Create view
2. Inserting Rows into a View
3. Deleting Rows from a View
4. Update View
5. Delete View

Description:

Views are virtual tables that allow users to structure data in a way that users find intuitive. It restricts access to data so the user can see and (sometimes) modify exactly what they need. A view is a logical table based on a table or another view. A view contains no data of its own but is like a window through which data from tables can be viewed or changed.

Creating Views:

Syntax

```
create view view_name<as> select column1, column2 ..... from table_name where [condition];
```

Inserting Rows into a View:

Rows of data can be inserted into a view. The same rules that apply to the insert command in DML are applied here.

Deleting Rows from a View / Delete view:

Rows of data can be deleted from a view. The same rules that apply to the delete command in

DML are applied here.

Updating a View:

The same rules that apply to the update command in DML are applied here. A

view can be updated under certain conditions:

1. The SELECT clause may not contain the keyword DISTINCT.
2. The SELECT clause may not contain set operators.
3. The SELECT clause may not contain an order by group by or having clause.
4. The FROM clause may not contain multiple tables.
5. The WHERE clause may not contain sub queries.

Sample Test Cases:

1. From the following table, create a view 'newyorkstaff' for those salespersons belonging to the city 'New York'.

Sample table: salesman

salesman_id	name	city	commission
5001	James Hoog	New York	0.15
5002	Nail Knite	Paris	0.13

Note: Table name and column names are case sensitive.

Output format

The output will display the View of salespersons belonging to the city 'New York'.

Solution:

```
create view newyorkstaff as  
select * from salesman where city='New York';  
select * from newyorkstaff;
```

2. From the following table, create a view 'totalforday' to count the number of unique customers, compute the average and total purchase amount of customer orders by each date. Round off the average and sum values to 2 decimal places using round() and Give an Alice name to Average as AVG and for the sum as SUM.

Sample table: orders

ord_no	purch_amt	ord_date	customer_id	salesman_id
70001	150.5	2012-10-05	3005	5002
70009	270.65	2012-09-10	3001	5005

Note: Table name and column names are case sensitive.

Output column name

ord_date	dist_customer	AVG	SUM
----------	---------------	-----	-----

Solution:

create view totalfor day as

select ord_date as ord_date, count(distinct customer_id) as dist_customer,

ROUND(avg(purch_amt),2) as AVG, ROUND(sum(purch_amt),2) as SUM

from orders

group by ord_date;

3. From the following table, create a view 'gradecount' to count the number of customers in each grade.

Table Name: customer

customer_id	cust_name	city	grade	salesman_id
3002	Nick Rimando	New York	100	5001
3007	Brad Davis	New York	200	5001
3005	Graham Zusi	California	200	5002

Note: Table names and column names are case-sensitive.

Output should have following fields

grade	number
-------	--------

Solution:

create view gradecount(grade,number) as

select grade,

count(*) as number from customer

group by grade;

Result:

Thus, views operations are executed successfully.

EX.NO:7

**Practice of Practice of Procedural extensions
(Cursors, Triggers)**

Aim:

To create PL/SQL programs to implement Cursors and Triggers

Learning Outcomes:

After the completion of this experiment, student will be able to

- Understand the need and usage of cursors and Triggers

To implement the various procedures we need the following parameters,

1. IN parameter
2. OUT parameter
3. IN OUT parameter

Parameters:

1. **IN:** It specifies that a value for the argument must be specified when calling the procedure ie., used to pass values to a sub-program. This is the default parameter.
2. **OUT:** It specifies that the procedure passes a value for this argument back to it's calling environment after execution ie. used to return values to a caller of the sub-program.
3. **IN OUT:** It specifies that a value for the argument must be specified when calling the procedure and that procedure passes a value for this argument back to it's calling environment after execution.

SAMPLE TESTCASES:

CURSOR:

1. Given the Employees table, Create a Cursor to find the list of all employees in a department with department Id passes as an argument.

As Cursor can be implemented inside Stored Procedure, Create a procedure

Procedure Name: EmployeeList(IN dept_id INT, INOUT employee_list VARCHAR(2000))

Department ID should be passed as an argument. employee_list will have all the employee names corresponding to the particular department.

Table Name: Employees

EMPLOYEE_ID INT PRIMARY KEY,
EMP_NAME VARCHAR(100),
EMAIL VARCHAR(30),
PHONE_NUMBER VARCHAR(20),
HIRE_DATE DATE,

JOB_ID VARCHAR(10),
EMP_SALARY INTEGER,
MANAGER_ID INT,
DEPARTMENT_ID INT
Each Employee name should end with ;

Solution:

```
DELIMITER $$
CREATE PROCEDURE EmployeeList(
In dept_id INT, INOUT employee_list VARCHAR(2000))
BEGIN
    DECLARE v_finished INTEGER DEFAULT 0;
    DECLARE v_ename VARCHAR(100) DEFAULT "";
    DECLARE C1 CURSOR FOR
    SELECT EMP_NAME FROM Employees WHERE DEPARTMENT_ID = dept_id;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_finished = 1;
    OPEN C1;
    get_emp: LOOP
        FETCH C1 INTO v_ename;
        IF v_finished = 1 THEN
            LEAVE get_emp;
        END IF;
        SET employee_list = CONCAT(employee_list, v_ename, ";");
    END LOOP get_emp;
    CLOSE C1;
    END $$
DELIMITER ;

SET @employeeelist = "";
CALL EmployeeList(97, @employeeelist);
SELECT @employeeelist;
```

2. Given Employees and Department table, Create a Cursor to find the list of all employees in a department with department name passed as an argument.

As Cursor can be implemented inside Stored Procedure, Create a procedure

Procedure Name: EmployeeList(IN dept_name VARCHAR(30), INOUT employee_list VARCHAR(2000))

Department name should be passed as an argument. employee_list will have all the employee names corresponding to the particular department name.

Table Name: Employees

EMPLOYEE_ID INT PRIMARY KEY,
EMP_NAME VARCHAR(100),
EMAIL VARCHAR(30),
PHONE_NUMBER VARCHAR(20),
HIRE_DATE DATE,
EMP_SALARY INTEGER,
DEPARTMENT_ID INT

Table Name: Department

(DEPARTMENT_ID int primary key, DEPARTMENT_NAME VARCHAR(30));
DEPARTMENT_ID in Employees table is related to DEPARTMENT_ID in Department table.
Each Employee name should end with ;

Solution:

DELIMITER \$\$

CREATE PROCEDURE EmployeeList(

In dept_name VARCHAR(30), INOUT employee_list VARCHAR(2000))

BEGIN

DECLARE v_finished INTEGER DEFAULT 0;

DECLARE v_ename VARCHAR(100) DEFAULT "";

DECLARE C1 CURSOR FOR

**SELECT EMP_NAME FROM Employees E JOIN Department D ON
E.DEPARTMENT_ID = D.DEPARTMENT_ID WHERE D.DEPARTMENT_NAME =
dept_name;**

DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_finished = 1;

OPEN C1;

get_emp: LOOP

FETCH C1 INTO v_ename;

IF v_finished = 1 THEN

LEAVE get_emp;

END IF;

SET employee_list = CONCAT(employee_list, v_ename, ";");

END LOOP get_emp;

CLOSE C1;

END \$\$

DELIMITER ;

SET @employeeelist = "";

CALL EmployeeList('Training', @employeeelist);

SELECT @employeeelist;

3. Given Product table, Implement a Cursor inside Stored Procedure to count how many products(product_name) of each product type exists with product type passed as a input argument to the procedure.

As Cursor can be implemented inside Stored Procedure, Create a procedure

Procedure Name:

ProductCount(IN in_prod_type VARCHAR(30))

Product type should be passed as an argument.

Table Name: Product

product_id int

product_type varchar(30)

product_name varchar(30)

Count will be displayed with header name ProductCount

Solution:

DELIMITER \$\$

CREATE PROCEDURE ProductCount(

IN in_prod_type VARCHAR(30))

BEGIN

DECLARE v_finished INTEGER DEFAULT 0;

DECLARE v_prod_type VARCHAR(30) DEFAULT "";

DECLARE v_prod_count INTEGER DEFAULT 0;

DECLARE C1 CURSOR FOR

SELECT product_type, count(*) FROM Product GROUP BY 1;

DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_finished = 1;

OPEN C1;

get_emp: LOOP

FETCH C1 INTO v_prod_type, v_prod_count;

IF v_finished = 1 THEN

LEAVE get_emp;

END IF;

IF v_prod_type = in_prod_type THEN

SELECT v_prod_count as ProductCount;

END IF;

END LOOP get_emp;

CLOSE C1;

END \$\$

DELIMITER ;

CALL ProductCount('Hygiene');

TRIGGERS:

4. AFTER INSERT TRIGGER

Create a trigger named `product_availability` that inserts mapping records into the `products_to_stores` table. This trigger is used to enforce business logic; in particular, it helps define the product availability for the different stores. When an item is being inserted into the `products` table, the trigger will check the `availability` field. If `availability` column is marked with the `LOCAL` value, the product will be made available in one store only. (We need to insert a new row in `products_to_stores` table with the new product id and `store_id` as 1.)

Example:

```
INSERT INTO products_to_stores (product_id, store_id) VALUES (new product id, 1);
```

Any other value will instruct the trigger to make the product available to the two stores that we created earlier.

(We need to insert 2 new rows in `products_to_stores` table with the new product id and `store_id` as 1 as one row and new product id and `store_id` 2 as second row)

Main Table Details:

stores	
store_id (PK)	BIGINT Auto increment
store_name	VARCHAR(50)

products	
product_id (PK)	BIGINT Auto increment
product_name	VARCHAR(40)
cost_price	DOUBLE
retail_price	DOUBLE
availability	VARCHAR(5)

products_to_stores Table:

products_to_stores	
ref_id (PK)	BIGINT Auto increment
product_id	BIGINT
store_id	BIGINT

Solution:

delimiter %

create trigger product_availability

after insert on products for each row

begin

if(new.availability='LOCAL') then

insert into products_to_stores(product_id,store_id)values(new.product_id,1);

else

insert into products_to_stores(product_id,store_id)values(new.product_id,1);

insert into products_to_stores(product_id,store_id)values(new.product_id,2);

end if;

end%

```
INSERT INTO products (product_name, cost_price, retail_price, availability) VALUES
('BLUETOOTH KEYBOARD', '17.60', '23.30','LOCAL');
INSERT INTO products (product_name, cost_price, retail_price, availability) VALUES
('DVB-T2 RECEIVE', '49.80', '53.40','ALL');
```

```
SELECT * FROM products_to_stores;
```

5. AFTER DELETE TRIGGER

Create a trigger named product_archiver that inserts a new row in archived_products table after we delete a particular product in products table.

Main Table Details:

stores	
store_id (PK)	BIGINT Auto increment
store_name	VARCHAR(50)

products	
product_id (PK)	BIGINT Auto increment
product_name	VARCHAR(40)
cost_price	DOUBLE
retail_price	DOUBLE
availability	VARCHAR(5)

Solution:

```
delimiter &
create trigger product_archiver
after delete on products
for each row
begin
insert into archived_products values(old.product_id,old.product_name,
old.cost_price,old.retail_price,old.availability);
end&
```

```
DELETE FROM products WHERE product_id=1;
```

```
SELECT * FROM archived_products;
```

6. Before Update Trigger

Create a trigger called 'account_balance_validator' which should be triggered before update operation taking place on customer_accounts table.

If customer account balance is updated the amount which is less than 0, then the account balance should be updated as 0.

Table Name: customer_accounts

customer_accounts	
cust_id (PK)	INT
cust_name	VARCHAR(60)
account_no	VARCHAR(60)
account_balance	INT

Solution:

```
delimiter $
create trigger account_balance_validator
before update on
customer_accounts for each row
begin
if new.account_balance < 0 then
set new.account_balance=0 ;
end if;
end$
```

```
UPDATE customer_accounts
SET
account_balance = -56
WHERE cust_id = 3;
```

```
SELECT * FROM customer_accounts;
```

Result:

Thus the PL/SQL programs were successfully executed.

Ex.No. 8**Application Development using front end tools****Aim :**

To implement Student Management system using Java with mysql

Procedure

- **Java Project Setup**
 - Create a new Java project in your favorite IDE and add the MySQL JDBC driver to your project's class path.
- **Database Setup**
 - First, let's create a database and a table to store our registration data

To connect Java with a database using JDBC (Java Database Connectivity), you need to follow these steps:

1. **Choose a JDBC Driver:** Select the appropriate JDBC driver for your database.
2. **Include the JDBC Driver in Your Project:** Add the driver JAR file to your project's classpath or specify it as a dependency in your project configuration file.
3. **Load the JDBC Driver:** Use `Class.forName()` to load the JDBC driver class into memory.
4. **Establish a Connection:** Use `DriverManager.getConnection()` to establish a connection to the database, providing the JDBC URL and authentication credentials.
5. **Create a Statement:** Create a `Statement` or `PreparedStatement` object to execute SQL queries.
6. **Execute SQL Queries:** Use `executeQuery()` to execute `SELECT` queries or `executeUpdate()` for `INSERT`, `UPDATE`, and `DELETE` queries.
7. **Process the Results:** If executing a `SELECT` query, process the results using a `ResultSet` object.
8. **Close the Connection:** Close the `Connection`, `Statement`, and `ResultSet` objects to release resources.

Program:

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.sql.*;

class RegistrationForm extends JFrame {

    private JTextField firstNameField, lastNameField, emailField;
    private JRadioButton maleRadioButton, femaleRadioButton;
    private JButton registerButton, viewAllButton, updateButton, deleteButton ;
    private JTextField rollNoField;

    public RegistrationForm() {
        setTitle("Registration Form");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setSize(400, 450);
        setLocationRelativeTo(null);

        JPanel panel = new JPanel();
        panel.setBackground(new Color(35, 100, 98));
        panel.setLayout(null);
        add(panel);

        JLabel titleLabel = new JLabel("Registration Form");
        titleLabel.setForeground(new Color(235, 208, 206));
        titleLabel.setFont(new Font("Arial", Font.CENTER_BASELINE, 20));
        titleLabel.setHorizontalAlignment(JLabel.CENTER);
        titleLabel.setBounds(100, 10, 200, 30);
        panel.add(titleLabel);

        // First Name
        JLabel firstNameLabel = new JLabel("First Name");
        firstNameLabel.setForeground(new Color(149, 151, 166));
        firstNameLabel.setBounds(20, 50, 100, 30);
        panel.add(firstNameLabel);

        firstNameField = new JTextField();
        firstNameField.setBounds(20, 75, 150, 30);
        panel.add(firstNameField);

        // Last Name
        JLabel lastNameLabel = new JLabel("Last Name");
        lastNameLabel.setForeground(new Color(149, 151, 166));
        lastNameLabel.setBounds(200, 50, 100, 20);
        panel.add(lastNameLabel);

        lastNameField = new JTextField();
        lastNameField.setBounds(200, 75, 150, 30);
```

```
panel.add(lastNameField);

// Email
JLabel emailLabel = new JLabel("Email");
emailLabel.setForeground(new Color(149, 151, 166));
emailLabel.setBounds(20, 110, 100, 20);
panel.add(emailLabel);

emailField = new JTextField();
emailField.setBounds(20, 135, 330, 30);
panel.add(emailField);

// Gender
JLabel genderLabel = new JLabel("Gender");
genderLabel.setForeground(new Color(149, 151, 166));
genderLabel.setBounds(20, 170, 100, 20);
panel.add(genderLabel);

JLabel rollNoLabel = new JLabel("Roll No");
rollNoLabel.setForeground(new Color(149, 151, 166));
rollNoLabel.setBounds(20, 320, 100, 30);
panel.add(rollNoLabel);

rollNoField = new JTextField();
rollNoField.setBounds(20, 355, 150, 30);
panel.add(rollNoField);

maleRadioButton = new JRadioButton("Male");
maleRadioButton.setBackground(Color.yellow);
femaleRadioButton = new JRadioButton("Female");
femaleRadioButton.setBackground(Color.yellow);
maleRadioButton.setBounds(100, 185, 70, 20);
femaleRadioButton.setBounds(200, 185, 80, 20);
ButtonGroup genderGroup = new ButtonGroup();
genderGroup.add(maleRadioButton);
genderGroup.add(femaleRadioButton);
panel.add(maleRadioButton);
panel.add(femaleRadioButton);

// Register Button
registerButton = new JButton("Register Now");
registerButton.setBounds(20, 230, 150, 30);
panel.add(registerButton);

// View All Button
viewAllButton = new JButton("View All");
viewAllButton.setBounds(190, 230, 150, 30);
panel.add(viewAllButton);

// Update Button
updateButton = new JButton("Update");
```

```

updateButton.setBounds(20, 280, 150, 30);
panel.add(updateButton);

// Delete Button
deleteButton = new JButton("Delete");
deleteButton.setBounds(190, 280, 150, 30);
panel.add(deleteButton);

// Action Listeners
registerButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {

        register();
    }
});

viewAllButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        viewAll();
    }
});

updateButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        update();
    }
});

deleteButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        delete();
    }
});

setVisible(true);
}

private void register() {
    // Perform registration operation and save data to the database
    String firstName = firstNameField.getText();
    String lastName = lastNameField.getText();
    String email = emailField.getText();
    String gender = maleRadioButton.isSelected() ? "Male" : "Female";
    int roll_no = Integer.parseInt(rollNoField.getText());

    // Perform database insertion here

```

```

        try {
            Connection connection =
DriverManager.getConnection("jdbc:mysql://localhost:3306/university", "root", "password");
            PreparedStatement ps = connection.prepareStatement("SELECT * FROM
university.registrationform");
            ResultSet rs = ps.executeQuery();

            ps = connection.prepareStatement("INSERT INTO university.registrationform
VALUES(?,?,?,?,?)");
            ps.setString(1,firstName);
            ps.setString(2,lastName);
            ps.setString(3,email);
            ps.setString(4,gender);
            ps.setInt(5,roll_no);

            ps.execute();
            JOptionPane.showMessageDialog(this, "Registration Successful!");
            connection.close();
        } catch (SQLException ex) {
            JOptionPane.showMessageDialog(this, "Error: " + ex.getMessage());
        }
    }

    private void viewAll() {
        // Retrieve all records from the database and display them in the console
        try {
            Connection connection = DriverManager.getConnection("jdbc:mysql://localhost:3306/university",
"root", "password");
            Statement statement = connection.createStatement();
            ResultSet resultSet = statement.executeQuery("SELECT * FROM registrationForm");

            System.out.println("All Records:");
            while (resultSet.next()) {
                String firstName = resultSet.getString("first_name");
                String lastName = resultSet.getString("last_name");
                String email = resultSet.getString("email");
                String gender = resultSet.getString("gender");
                int roll_no = resultSet.getInt("roll_no");

                System.out.println("Name: " + firstName + " " + lastName + ", Email: " + email + ", Gender: "
+ gender + ", Roll No: " + roll_no);
            }
            connection.close();
        } catch (SQLException ex) {
            System.out.println("Error: " + ex.getMessage());
        }
    }

    private void update() {
        // Perform update operation on a selected record

```

```

String firstName = firstNameField.getText();
String lastName = lastNameField.getText();
String email = emailField.getText();
String gender = maleRadioButton.isSelected() ? "Male" : "Female";
int rollno = Integer.parseInt(rollNoField.getText());

// You can modify this query based on your database schema
String updateQuery = "UPDATE registrationForm SET first_name=?, last_name=?, email=?,
gender=? WHERE roll_no=?";

try (Connection connection =
DriverManager.getConnection("jdbc:mysql://localhost:3306/university", "root", "password");
    PreparedStatement preparedStatement = connection.prepareStatement(updateQuery)) {

    // Set parameters for the prepared statement
    preparedStatement.setString(1, firstName);
    preparedStatement.setString(2, lastName);
    preparedStatement.setString(3, email);
    preparedStatement.setString(4, gender);
    preparedStatement.setInt(5, rollno);

    // Execute the update query
    int rowsAffected = preparedStatement.executeUpdate();

    if (rowsAffected > 0) {
        JOptionPane.showMessageDialog(this, "Record updated successfully!");
    } else {
        JOptionPane.showMessageDialog(this, "No record found with the specified roll number.");
    }
} catch (SQLException ex) {
    JOptionPane.showMessageDialog(this, "Error: " + ex.getMessage());
}
}

private void delete() {
    // Perform delete operation on a selected record
    String rollno = rollNoField.getText();

    // You can modify this query based on your database schema
    String deleteQuery = "DELETE FROM registrationForm WHERE roll_no=?";

    try (Connection connection =
DriverManager.getConnection("jdbc:mysql://localhost:3306/university", "root", "password");
        PreparedStatement preparedStatement = connection.prepareStatement(deleteQuery)) {

        // Set the roll number parameter for the prepared statement
        preparedStatement.setString(1, rollno);

        // Execute the delete query
        int rowsAffected = preparedStatement.executeUpdate();

```

```

        if (rowsAffected > 0) {
            JOptionPane.showMessageDialog(this, "Record deleted successfully!");
        } else {
            JOptionPane.showMessageDialog(this, "No record found with the specified roll number.");
        }
    } catch (SQLException ex) {
        JOptionPane.showMessageDialog(this, "Error: " + ex.getMessage());
    }
}

public static void main(String[] args) {

    RegistrationForm form = new RegistrationForm();
}
}

CREATE DATABASE university;

USE university;

CREATE TABLE registrationForm (

    first_name VARCHAR(50),

    last_name VARCHAR(50),

    email VARCHAR(50),

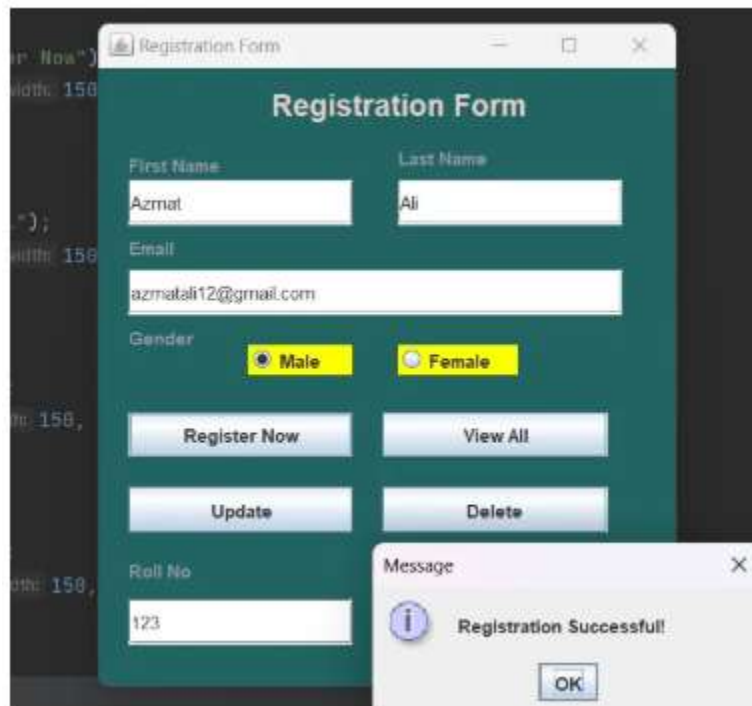
    gender VARCHAR(10),

    roll_no INT PRIMARY KEY

);

```

OUTPUT



The screenshot displays a web browser window titled "Registration Form". The form has a dark green header with the title "Registration Form". Below the header, there are input fields for "First Name" (containing "Azmat"), "Last Name" (containing "Ali"), and "Email" (containing "azmatali12@gmail.com"). There are two radio buttons for "Gender": "Male" (selected) and "Female". Below these are four buttons: "Register Now", "View All", "Update", and "Delete". At the bottom, there is a "Roll No" field containing "123". A small "Message" dialog box is open in the bottom right corner, displaying an information icon and the text "Registration Successful!" with an "OK" button.

Result:

Thus the Student Management System was successfully implemented.

Ex.No. 9 Document Database creation using MongoDB

Aim :

To implement Document Database creation using MongoDB

MongoDB Installation

Installation Method

Installs MongoDB on Windows using the default MSI installation wizard. Also install MongoDB on Windows by these other methods:

- [Install MongoDB using msiexec.exe.](#)
- [Install MongoDB from the zip file](#)

CREATE A DATABASE

There is no CREATE DATABASE statement in MongoDB like there is in SQL. To create a database in MongoDB, simply switch to a non-existent database, then insert data into it.

To switch databases, run the `use` statement. If the database doesn't already exist, it will be created:

```
use music
```

This results in the following message:

```
switched to db music
```

However, the database isn't actually created until you insert data into it:

```
db.artists.insert({ artistname: "The Tea Party" })
```

The above statement creates a collection and inserts a document into it.

It will generate the following message:


```
WriteResult({ "nInserted" : 1 })
```

You can see the database in your list of databases by issuing the following command:

```
show databases
```

Here's an example of the output:

```
local 0.000GB  
music 0.000GB  
test 0.005GB
```

In this case, three databases are displayed, one of which is our newly created database (music).

You can also run the following line to view the contents of your database:

```
db.artists.find()
```

Which should result in output like this:

```
{ "_id" : ObjectId("5780fbf948ef8c6b3ffb0149"), "artistname" : "The Tea Party" }
```

CREATE A COLLECTION

Two ways to Create a Collection

Here are two ways of creating collections:

- You can create a collection on the fly when inserting a document (using the `insert()` method).
- You can also create a collection explicitly, using the `createCollection()` method.

On the Fly

When you use the `insert()` method to insert a document, you specify the collection that the document will be inserted into. If the collection doesn't already exist, it will be created.

This is the method that we used previously when we created our `artists` collection while inserting a document.

Here's the code that we used:

```
db.artists.insert({ artistname: "The Tea Party" })
```

In this case, the `artists` collection didn't previously exist so it was created for us.

Using the `createCollection()` Method

You can also create collections using the `createCollection()` method. This allows you to create a collection without inserting a document.

Here's an example of using the `createCollection()` method:

```
db.createCollection("producers")
```

With Options

You can also specify options for the collection by using the `db.createCollection(name, options)` syntax.

Here's an example:

```
db.createCollection("log", { capped : true, size : 4500500, max : 4000 } )
```

CREATE A DOCUMENT

MongoDB provides the following three methods for inserting documents into a database:

- `insert()`
- `insertOne()`
- `insertMany()`

The `insert()` Method

The `insert()` method inserts one or more documents into a collection. Each document is provided as a parameter. The collection name is prepended to the `insert()` method.

Here's the syntax for inserting a single document:

```
db.collectionName.insert({ name: "value" })
```

In the above example, the document consists of { name: "value" }. This is a JSON document. JSON documents consist of one or more name/value pairs, enclosed in curly braces {}.

MongoDB uses JSON documents to store data, so that's why we insert documents in this format.

Let's add another document to our database:

```
db.artists.insert({ artistname: "Jorn Lande" })
```

This inserts a document with { artistname: "Jorn Lande" } as its contents.

Now, if we search the artists collection, we will see two documents (including the one we created [previously](#)):

```
> db.artists.find()

{ "_id" : ObjectId("5780fbf948ef8c6b3ffb0149"), "artistname" : "The Tea Party" }

{ "_id" : ObjectId("5781c9ac48ef8c6b3ffb014a"), "artistname" : "Jorn Lande" }
```

Note that MongoDB has created an `_id` field for the documents. If you don't specify one, MongoDB will create one for you. However, you can provide this field when doing the insert if you prefer to have control over the value of the `_id` field.

```
db.artists.insert({ _id: 1, artistname: "AC/DC" })
```

Result:

```
> db.artists.find()

{ "_id" : ObjectId("5780fbf948ef8c6b3ffb0149"), "artistname" : "The Tea Party" }

{ "_id" : ObjectId("5781c9ac48ef8c6b3ffb014a"), "artistname" : "Jorn Lande" }

{ "_id" : 1, "artistname" : "AC/DC" }
```

The `_id` that MongoDB provides is a 12-byte ObjectId value. It is made up of the following values;

- a 4-byte value representing the seconds since the Unix epoch,
- a 3-byte machine identifier,
- a 2-byte process id, and
- a 3-byte counter, starting with a random value.

Create Multiple Documents

You can insert multiple documents within a single `insert()` method.

In this example, we insert three documents:

```
db.artists.insert([
  { artistname: "The Kooks" },
  { artistname: "Bastille" },
  { artistname: "Gang of Four" }
])
```

Note that the documents are provided as an array. The documents are enclosed in square brackets [], and they are separated by commas.

Running the above code results in the following message:

```
BulkWriteResult({
  "writeErrors" : [ ],
  "writeConcernErrors" : [ ],
  "nInserted" : 3,
  "nUpserted" : 0,
  "nMatched" : 0,
  "nModified" : 0,
  "nRemoved" : 0,
  "upserted" : [ ]
})
```

Embedded Documents

A document can contain other documents, arrays, and arrays of documents.

You can also provide multiple name/value pairs within a document by separating them with a comma.

```
db.artists.insert({
  artistname : "Deep Purple",
  albums : [
    {
      album : "Machine Head",
      year : 1972,
      genre : "Rock"
    },
    {
      album : "Stormbringer",
      year : 1974,
      genre : "Rock"
    }
  ]
})
```

Result:

```
WriteResult({ "nInserted" : 1 })
```

Parameters

The `insert()` method accepts the following parameters.

Parameter	Type	Description
<code>document</code>	document or array	A document or array of documents to insert into the collection (as in the above examples).
<code>writeConcern</code>	document	Optional parameter. This is a document expressing the write concern. A write concern describes the level of acknowledgement requested from MongoDB for write operations to a standalone mongod or to replica sets or to sharded clusters.
<code>ordered</code>	boolean	Optional parameter. If the value is set to true , MongoDB will perform an ordered insert of the documents in the array, and if an error occurs with one of documents, MongoDB will return without processing the remaining documents in the array. If the value is set to false , MongoDB will perform an unordered insert, and if

		an error occurs with one of documents, the remaining documents in the array will continue to be processed.
--	--	--

The insertOne() Method

You can also use the `insertOne()` method to insert a single document into a collection:

```
db.musicians.insertOne({_id: 1, name: "Ian Gillan", instrument: "Vocals" })
```

Here, we've specified a non-existent collection. As with the `insert()` method, the specified collection will be created if it doesn't already exist.

You'll notice that the output is different to when you use the `insert()` method:

```
{ "acknowledged" : true, "insertedId" : 1 }
```

Embedded Documents

As with `insert()`, you can insert embedded documents and arrays of documents:

```
db.artists.insertOne({
  artistname : "Miles Davis",
  albums : [
    {
      album : "Kind of Blue",
      year : 1959,
      genre : "Jazz"
    },
    {
      album : "Bitches Brew",
      year : 1970,
      genre : "Jazz"
    }
  ]
})
```

Result:

```
{
  "acknowledged" : true,
```

```
"insertedId" : ObjectId("578214f048ef8c6b3ffb0159")
}
```

The insertMany() Method

As the name suggests, you can use `insertMany()` to insert multiple documents:

```
db.musicians.insertMany(
[
  { _id: 2, name: "Ian Paice", instrument: "Drums", born: 1948 },
  { _id: 3, name: "Roger Glover", instrument: "Bass", born: 1945 },
  { _id: 4, name: "Steve Morse", instrument: "Guitar", born: 1954 },
  { _id: 5, name: "Don Airey", instrument: "Keyboards", born: 1948 },
  { _id: 6, name: "Jeff Martin", instrument: "Vocals", born: 1969 },
  { _id: 7, name: "Jeff Burrows", instrument: "Drums", born: 1968 },
  { _id: 8, name: "Stuart Chatwood", instrument: "Bass", born: 1969 },
]
)
```

Again, the output when using `insertMany()` is different than if you'd inserted multiple documents using the `insert()` method:

```
{
  "acknowledged" : true,
  "insertedIds" : [
    2,
    3,
    4,
    5,
    6,
    7,
    8
  ]
}
```

```
}
```

Embedded Documents

```
db.artists.insertMany([
  {
    artistname : "Robben Ford",
    albums : [
      {
        album : "Bringing it Back Home",
        year : 2013,
        genre : "Blues"
      },
      {
        album : "Talk to Your Daughter",
        year : 1988,
        genre : "Blues"
      }
    ]
  }, {
    artistname : "Snoop Dogg",
    albums : [
      {
        album : "Tha Doggfather",
        year : 1996,
        genre : "Rap"
      },
      {
        album : "Reincarnated",
        year : 2013,
        genre : "Reggae"
      }
    ]
  }
])
```

Result:

```
{
  "acknowledged" : true,
  "insertedIds" : [
```



```
        ObjectId("578217c248ef8c6b3ffb015a"),  
        ObjectId("578217c248ef8c6b3ffb015b")  
    ]  
}
```

Result:

Thus the above program was successfully executed.

Aim:

To Study the Cloud storage.

Cloud Storage

Cloud Storage is a mode of computer data storage in which digital data is stored on servers in off-site locations. The servers are maintained by a third-party provider who is responsible for hosting, managing, and securing data stored on its infrastructure. The provider ensures that data on its servers is always accessible via public or private internet connections.

Cloud Storage enables organizations to store, access, and maintain data so that they do not need to own and operate their own data centers, moving expenses from a capital expenditure model to operational. Cloud Storage is scalable, allowing organizations to expand or reduce their data footprint depending on need.

Google Cloud provides a variety of scalable options for organizations to store their data in the cloud.

Working of Cloud Storage

Cloud Storage uses remote servers to save data, such as files, business data, videos, or images. Users upload data to servers via an internet connection, where it is saved on a virtual machine on a physical server. To maintain availability and provide redundancy, cloud providers will often spread data to multiple virtual machines in data centers located across the world. If storage needs increase, the cloud provider will spin up more virtual machines to handle the load. Users can access data in Cloud Storage through an internet connection and software such as web portal, browser, or mobile app via an application programming interface (API).

Cloud Storage is available in four different models:

Public

Public Cloud Storage is a model where an organization stores data in a service provider's data centers that are also utilized by other companies. Data in public Cloud Storage is spread across multiple regions and is often offered on a subscription or pay-as-you-go basis. Public Cloud Storage is considered to be "elastic" which means that the data stored can be scaled up or down depending on the needs of the organization. Public cloud providers typically make data available from any device such as a smartphone or web portal.

Private

Private Cloud Storage is a model where an organization utilizes its own servers and data centers to store data within their own network. Alternatively, organizations can deal with cloud service providers to provide dedicated servers and private connections that are not shared by any other organization. Private clouds are typically utilized by organizations that require more control over their data and have stringent compliance and security requirements.

Hybrid

A hybrid cloud model is a mix of private and public cloud storage models. A hybrid cloud storage model allows organizations to decide which data it wants to store in which cloud. Sensitive data and data that must meet strict compliance requirements may be stored in a private cloud while less sensitive data is stored in the public cloud. A hybrid cloud storage model typically has a layer of orchestration to integrate between the two clouds. A hybrid cloud offers flexibility and allows organizations to still scale up with the public cloud if need arises.

Multicloud

A multicloud storage model is when an organization sets up more than one cloud model from more than one cloud service provider (public or private). Organizations might choose a multicloud model if one cloud vendor offers certain proprietary apps, an organization requires data to be stored in a specific country, various teams are trained on different clouds, or the organization needs to serve different requirements that are not stated in the servicers' Service Level Agreements. A multicloud model offers organizations flexibility and redundancy.

Advantages of Cloud Storage

Total cost of ownership

Cloud Storage enables organizations to move from a capital expenditure to an operational expenditure model, allowing them to adjust budgets and resources quickly.

Elasticity

Cloud Storage is elastic and scalable, meaning that it can be scaled up (more storage added) or down (less storage needed) depending on the organization's needs.

Flexibility

Cloud Storage offers organizations flexibility on how to store and access data, deploy and budget resources, and architect their IT infrastructure.

Security

Most cloud providers offer robust security, including physical security at data centers and cutting edge security at the software and application levels. The best cloud providers offer [zero trust architecture](#), identity and [access management](#), and [encryption](#).

Sustainability

One of the greatest costs when operating on-premises data centers is the overhead of energy consumption. The best cloud providers [operate on sustainable energy](#) through renewable resources.

Redundancy

Redundancy (replicating data on multiple servers in different locations) is an inherent trait in public clouds, allowing organizations to recover from disasters while maintaining business continuity.

Disadvantages of Cloud Storage

Compliance

Certain industries such as finance and healthcare have stringent requirements about how data is stored and accessed. Some public cloud providers [offer tools to maintain compliance](#) with applicable rules and regulations.

Latency

Traffic to and from the cloud can be delayed because of network traffic congestion or slow internet connections.

Control

Storing data in public clouds relinquishes some control over access and management of that data, entrusting that the cloud service provider will always be able to make that data available and maintain its systems and security.

Outages

While public cloud providers aim to ensure continuous availability, outages sometimes do occur, making stored data unavailable.

Use of Cloud Storage

Cloud Storage provides several use cases that can benefit individuals and organizations. Whether a person is storing their family budget on a spreadsheet, or a massive organization is saving years of financial data in a highly secure database, Cloud Storage can be used for saving digital data of all kinds for as long as needed.

Backup

Data backup is one of the simplest and most prominent uses of Cloud Storage. Production data can be separated from backup data, creating a gap between the two that protects organizations in the case of a cyber threat such as ransomware. Data backup through Cloud Storage can be as simple as saving files to a digital folder such as Google Drive or using block storage to maintain gigabytes or more of important business data.

Archiving

The ability to archive old data has become an important aspect of Cloud Storage, as organizations move to digitize decades of old records, as well as hold on to records for governance and compliance purposes. Google Cloud offers several tiers of storage for archiving data, including [coldline storage](#) and [archival storage](#), that can be accessed whenever an organization needs them.

Disaster recovery

A disaster—natural or otherwise— that wipes out a data center or old physical records needs not be the business-crippling event that it was in the past. Cloud Storage allows for disaster recovery so that organizations can continue with their business, even when times are tough.

Data processing

As Cloud Storage makes digital data immediately available, data becomes much more useful on an ongoing basis. Data processing, such as analyzing data for business intelligence or

applying machine learning and artificial intelligence to large datasets, is possible because of Cloud Storage.

Content delivery

With the ability to save copies of media data, such as large audio and video files, on servers dispersed across the globe, media and entertainment companies can serve their audience low-latency, always available content from wherever they reside.

Types of Cloud Storage

Cloud Storage comes in three different types: **object**, **file**, and **block**.

Object

Object storage is a data storage architecture for large stores of unstructured data. It designates each piece of data as an object, keeps it in a separate storehouse, and bundles it with metadata and a unique identifier for easy access and retrieval.

File

File storage organizes data in a hierarchical format of files and folders. File storage is common in personal computing where data is saved as files and those files are organized in folders. File storage makes it easy to locate and retrieve individual data items when they are needed. File storage is most often used in directories and data repositories.

Block

Block storage breaks data into blocks, each with a unique identifier, and then stores those blocks as separate pieces on the server. The cloud network stores those blocks wherever it is most efficient for the system. Block storage is best used for large volumes of data that require low latency such as workloads that require high performance or databases.

Result:

Thus the Study on Cloud storage was completed.

Objectives:

To implement Payroll Processing System using SQL.

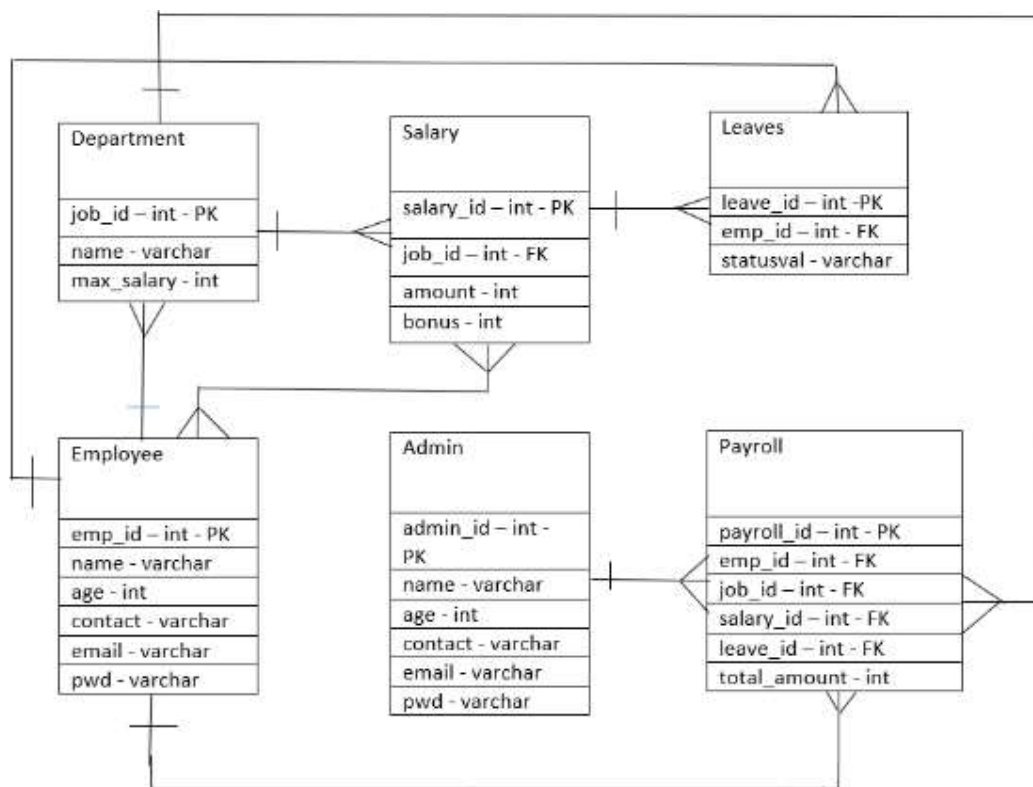
Learning Outcomes:

After the completion of this experiment, student will be able to

- Understand the application of various sql commands in the table.
- Summarize data from various tables which can be used to generate reports.

Problem Statement:

Given the schema of a Payroll Processing System,



1. Write a query to count the number of Employee with age<35.

Input format

No console input.

Output format

The output prints the count of the Employee with age<35.

Output Header: AgeCount

Solution:

```
SELECT COUNT(age) as AgeCount FROM Employee WHERE age<35;
```

2. Write a query to find the sum of the bonus given to all the employees from the salary table. Write a query to find the average age of all the employees from the employee table.

Input format

No console input.

Output format

The output prints the sum of the bonus given to all the employees from the salary table.

The output prints the average age of all the employees from the employee table.

Output Header: Bonus, Age

Sample output

Bonus
18000
Age
23.9000

Solution:

```
select sum(bonus) as Bonus from Salary;
```

```
select avg(age) as Age from Employee;
```

3. Given the schema of a Payroll Processing System, write a query to display the name and age of the Employee order by age and display the names and max_salary of employees from the Department table order by max_salary in descending order.

Input format

No console input.

Output format

The output prints the name and age of the Employee order by age and displays the names and max_salary of employees from the Department table order by max_salary in descending order.

Sample output

name	age
Eva	20

Solution

```
SELECT name, age FROM Employee ORDER BY age;
```

```
SELECT name, max_salary FROM Department ORDER BY max_salary DESC;
```

4. Write a query to count the leave_id based on statusval in the Leaves table.

Input format

No console input.

Output format

The output prints the count of leave_id based on status.

Output Header: LeaveID Status

Solution

```
SELECT COUNT(leave_id) as LeaveID, statusval as Status FROM Leaves GROUP BY statusval;
```

5. Write a query to perform the right join operation on Employee and Leaves table on emp_id.

Input format

No console input.

Output format

The output prints the result of the join operation.

The output displays the name.

Sample output:

```
name
Alice
Alice
```

Solution

```
SELECT name FROM Employee RIGHT JOIN Leaves on Employee.emp_id = Leaves.emp_id;
```

RESULT:

Result:

Thus Payroll Processing System was implemented successfully.

EX.NO: 11 b

Railway Reservation System

Objectives:

To implement Railway Reservation System using SQL.

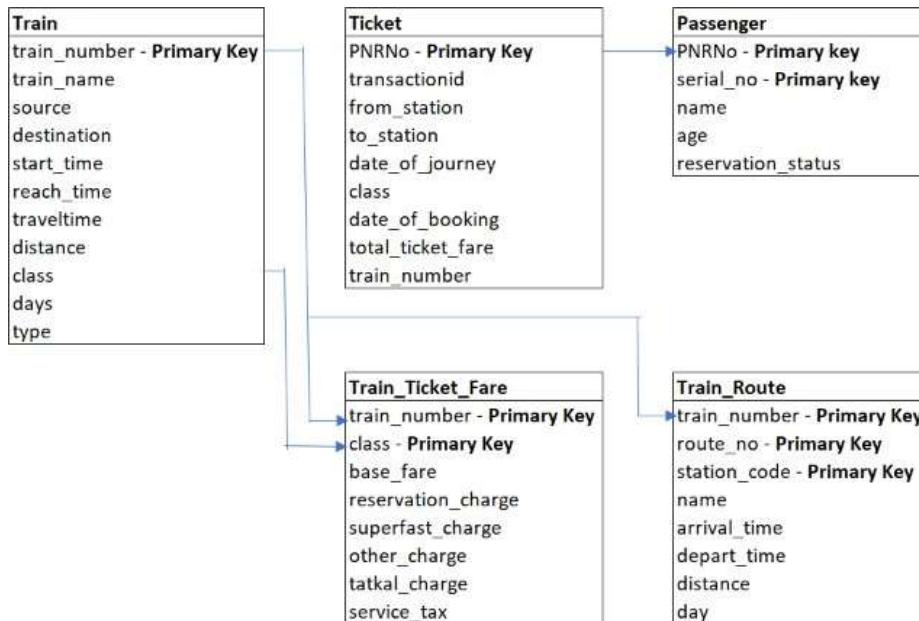
Learning Outcomes:

After the completion of this experiment, student will be able to

- Understand the application of various sql commands in the table.
- Summarize data from various tables which can be used to generate reports.

Problem Statement:

Given the schema of Railway Reservation System



1. write a query to list all the trains in ascending order of train number.

Note:

Refer the table **Train** from the given schema.

Table names are case sensitive.

Refer Output Format section for the output header names.

Input format

Refer above table

Output format

The output should have the below header for the query to be considered.

train_number, train_name

Solution

```
SELECT train_number, train_name
FROM Train
```

ORDER BY train_number;

2. Write a query to list the senior citizen passenger details. Sort the output by ascending order of passenger names.

Note:

Refer the table **Passenger** from the given schema.

Age of senior citizen should be greater than or equal to 60.

Input format

Refer above table

Output format

The output should have the below header for the query to be considered.

PNRNo, serial_no, name, age, reservation_status

Solution

```
SELECT *  
FROM Passenger  
WHERE age >= 60  
ORDER BY name;
```

3. write a query to find the train names that are from Chennai to Mumbai, but do not have the source or destination in its name (train_name). Sort the output by descending order of train name.

Note:

Refer the table **Train** from the given schema.

Table names are case sensitive.

Refer Output Format section for the output header names.

Input format

Refer above table

Output format

The output should have the below header for the query to be considered.

train_name

Solution

```
SELECT train_name  
FROM Train  
WHERE source = 'Chennai' AND destination = 'Mumbai'  
AND train_name NOT LIKE '%Chennai%' AND train_name NOT LIKE '%Mumbai%'  
ORDER BY train_name desc;
```

4. write a query to find the passengers(PNRNo, name, age) whose date of journey is 30 days from today. (Consider today's date as '2019-09-01' (YYYY-MM-DD)). Sort the output by ascending order of PNRNo and passenger name.

Note:

Refer the table Ticket and Passenger from the given schema.

Table names are case sensitive.

Refer Output Format section for the output header names.

Input format

Refer above table

Output format

The output should have the below header for the query to be considered.
PNRNo, name, age

Solution

```
SELECT P.PNRNo, P.name, P.age
FROM
Ticket T,
Passenger P
WHERE T.PNRNo = P.PNRNo
AND T.date_of_journey = DATE_ADD("2019-09-01", INTERVAL 30 DAY)
ORDER BY 1,2;
```

5. write a query to print the station codes replacing 'M' with 'K' along with the station name.

Note:

Refer the table **Train_Route** from the given schema.

Table names are case sensitive.

Refer Output Format section for the output header names.

Input format

Refer above table

Output format

The output should have the below header for the query to be considered.

station_code, name

Solution

```
SELECT replace(station_code,'M','K') as station_code, name
FROM Train_Route
ORDER BY 1;
```

RESULT:

Thus the Railway Reservation System was implemented successfully.

Objectives:

To implement Bank Management System using SQL.

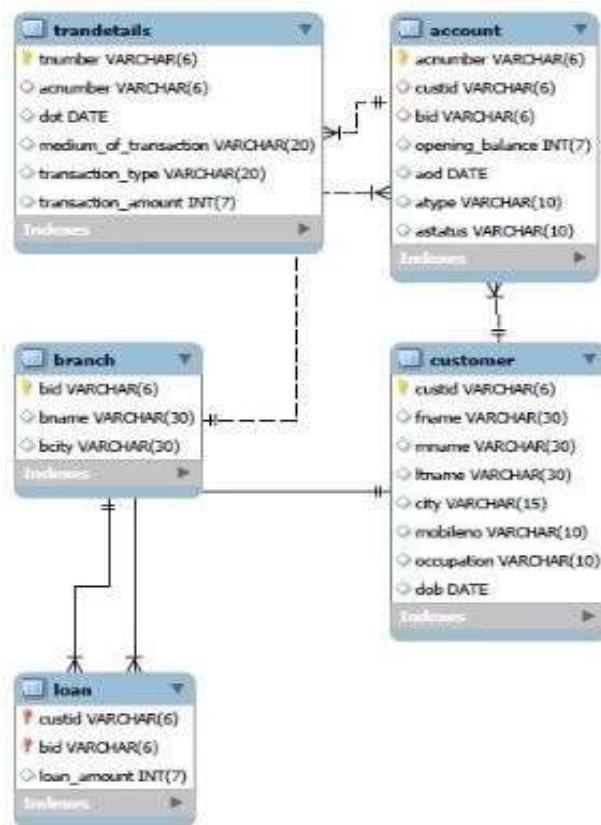
Learning Outcomes:

After the completion of this experiment, student will be able to

- Understand the application of various sql commands in the table.
- Summarize data from various tables which can be used to generate reports.

Problem Statement:

Given the schema of a Bank Management system



1. Write a query to display the custid, fname and customer's dob. Display in sorted order of date of birth year and within that sort by firstname.

Note:

The required input tables are populated in the back end.

The table names are case-sensitive.

Refer to the schema and the output section for further clarifications.

Input format

No console input.

Output format

The output prints the custid, fname and customer's dob in sorted order of date of birth year and within that in the sorted order of firstname.

Solution

```
select custid,fname,dob from customer order by dob,fname;
```

2. Write a query to display the customer's number, first name, and middle name. The customer's who don't have a middle name, for them display the last name. Give the alias name as Cust_Name.

Note:

The required input tables are populated in the back end.

The table names are case-sensitive.

Refer to the schema and the output section for further clarifications.

Input format

No console input.

Output format

The output prints the custid and Cust_Name.

Solution

```
select custid,concat(concat(fname,' '),coalesce(mname,ltname))as Cust_Name from customer;
```

3. Write a query to display the customer number, customer first name, account number for the customers whose accounts were created after the 15th of any month.

Note:

The required input tables are populated in the back end.

The table names are case-sensitive.

Refer to the schema and the output section for further clarifications.

Input format

No console input.

Output format

The output prints the result.

Sample output:

custid	fname	acnumber
CCC101	Alice	A00101

Solution

```
select a.custid,c.fname,a.acnumber from account a,customer c where a.custid=c.custid and day(aod)>15;
```

4. Write a query to display the customer's custid,firstname, city and account number who are not a teacher or tailor.

Note:

The required input tables are populated in the back end.

The table names are case-sensitive.

Refer to the schema and the output section for further clarifications.

Input format

No console input.

Output format

The output prints the custid,first name, city, and account number of customers who are not a teacher or tailor.

Solution

```
select distinct c.custid,c.fname,c.city,a.acnumber from customer c,account a where a.custid=c.custid and occupation not in
```